

Guía del backend

Todos los archivos y sus líneas importantes

Revisión: 30/09/2026 11:24

37 archivos · Java 21 · Spring Boot · PostgreSQL

Actualización: contactos como personas compartidas, mínimo uno sin máximo; parentesco por relación, distinción de titulares y JWT de una hora.

Cada capítulo identifica ruta, responsabilidad y líneas verificadas del archivo actual. Los números cuentan también líneas vacías. Si una línea larga se divide visualmente, solo el primer renglón lleva número.

El código se reproduce con numeración. Contraseña de base, clave JWT y hashes se omiten en la copia. Las sangrías excesivas de SQL se reducen sin alterar números de línea. README y lanzadores Maven se muestran con extractos.

Método: lectura estática. Maven reportó 24 pruebas, cero fallos y siete integraciones omitidas por no configurar una base desechable; no se modificó `nomina_db`.

Funcionamiento y cambios actuales

Frontend → filtros → controlador → servicio → repositorio → PostgreSQL. El DTO transporta datos; la entidad modela su persistencia.

AuthController compara BCrypt. JwtUtil emite tokens de una hora y JwtFilter establece autenticación; solo POST /api/auth/login es público.

guardar exige mínimo un contacto, comunicaciones completas y máximo 20 titulares activos. Crea personas o reutiliza IDs, vincula parentescos y persiste transaccionalmente. No hay bloqueo concurrente del contador.

actualizar exige correo y teléfono principal y secundario. Crea las posiciones faltantes y modifica las dos primeras. No cambia contactos: para ello se utiliza la ruta específica de contactos.

La respuesta contiene comunicaciones, lista completa de contactos con sus IDs y campos singulares del primero. PUT de contactos reemplaza los vínculos sin eliminar personas compartidas.

PerfilTitular distingue titulares de contactos; los contactos nuevos requieren los campos personales del esquema. Los vínculos existentes conservan sus datos compartidos.

Validación local: Maven reportó 24 pruebas, cero fallos y siete integraciones omitidas por no configurar una base PostgreSQL desechable.

El README contiene el ejemplo JSON actualizado, rutas y configuración. Para encontrar una referencia, abrir la ruta del capítulo en IntelliJ y pulsar Ctrl+G con el número indicado.

Índice navegable

- 01. `src/main/java/com/example/demo/DemoApplication.java`
- 02. `src/main/java/com/example/demo/controller/AuthController.java`
- 03. `src/main/java/com/example/demo/controller/FormularioController.java`
- 04. `src/main/java/com/example/demo/controller/OcupacionController.java`
- 05. `src/main/java/com/example/demo/service/PersonaService.java`
- 06. `src/main/java/com/example/demo/config/SecurityConfig.java`
- 07. `src/main/java/com/example/demo/security/JwtUtil.java`
- 08. `src/main/java/com/example/demo/security/JwtFilter.java`
- 09. `src/main/java/com/example/demo/dto/FormularioDTO.java`
- 10. `src/main/java/com/example/demo/dto/ContactoDTO.java`
- 11. `src/main/java/com/example/demo/model/Persona.java`
- 12. `src/main/java/com/example/demo/model/PerfilTitular.java`
- 13. `src/main/java/com/example/demo/model/CatalogoOcupacion.java`
- 14. `src/main/java/com/example/demo/model/PersonaCorreo.java`
- 15. `src/main/java/com/example/demo/model/PersonaTelefono.java`
- 16. `src/main/java/com/example/demo/model/ContactoEmergencia.java`
- 17. `src/main/java/com/example/demo/model/Usuario.java`
- 18. `src/main/java/com/example/demo/repository/PersonaRepository.java`
- 19. `src/main/java/com/example/demo/repository/CatalogoOcupacionRepository.java`
- 20. `src/main/java/com/example/demo/repository/UsuarioRepository.java`
- 21. `src/main/resources/application.properties`
- 22. `src/main/resources/schema.sql`
- 23. `src/main/resources/data.sql`
- 24. `pom.xml`
- 25. `scripts/HashPassword.java`
- 26. `src/test/java/com/example/demo/service/RegistroCompletoTest.java`
- 27. `src/test/java/com/example/demo/DemoApplicationTests.java`
- 28. `.mvn/wrapper/maven-wrapper.properties`
- 29. `mvnw.cmd`
- 30. `mvnw`
- 31. `.gitignore`
- 32. `.gitattributes`
- 33. `README.md`
- 34. `src/main/java/com/example/demo/model/CatalogoParentesco.java`
- 35. `src/main/java/com/example/demo/repository/CatalogoParentescoRepository.java`
- 36. `src/main/java/com/example/demo/controller/ParentescoController.java`
- 37. `src/test/java/com/example/demo/security/JwtUtilTest.java`

01. DemoApplication.java

src/main/java/com/example/demo/DemoApplication.java

13 líneas originales. SHA-256 de revisión: 93ed6cd1967aa82c...

Punto de entrada del backend. Arranca Spring y descubre los componentes del paquete de aplicación.

Líneas importantes

Línea 6. Activa configuración automática y descubrimiento de componentes.

Línea 10. Inicia el contexto de aplicación y el servidor.

Código numerado

```
1 | package com.example.demo;
2 |
3 | import org.springframework.boot.SpringApplication;
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
5 |
6 | @SpringBootApplication
7 | public class DemoApplication {
8 |
9 |     public static void main(String[] args) {
10 |         SpringApplication.run(DemoApplication.class, args);
11 |     }
12 |
13 | }
```

02. AuthController.java

src/main/java/com/example/demo/controller/AuthController.java

61 líneas originales. SHA-256 de revisión: 1d12c1bec983b913...

Recibe las credenciales y genera el token. El login usa directamente UsuarioRepository, PasswordEncoder y JwtUtil.

Líneas importantes

Línea 14. Agrupa las rutas bajo /api/auth.

Línea 27. Define POST /api/auth/login.

Línea 29. Lee username del cuerpo JSON; la línea siguiente lee password.

Línea 34. Busca la cuenta. El bloque siguiente responde 404 si no existe.

Línea 43. Compara la contraseña recibida con BCrypt. Si falla se devuelve 401.

Línea 53. Emite un JWT con el nombre de usuario.

Línea 55. Devuelve message, username y token. No hay comprobación de rol ni de baja en el método.

Código numerado

```
1 | package com.example.demo.controller;
2 |
3 | import com.example.demo.model.Usuario;
4 | import com.example.demo.repository.UsuarioRepository;
5 | import com.example.demo.security.JwtUtil;
6 | import org.springframework.beans.factory.annotation.Autowired;
7 | import org.springframework.http.ResponseEntity;
8 | import org.springframework.security.crypto.password.PasswordEncoder;
9 | import org.springframework.web.bind.annotation.*;
10 |
11 | import java.util.Map;
12 |
13 | @RestController
14 | @RequestMapping("/api/auth")
15 | @CrossOrigin(origins = "http://localhost:4200")
16 | public class AuthController {
17 |
18 |     @Autowired
19 |     private UsuarioRepository usuarioRepository;
20 |
21 |     @Autowired
22 |     private PasswordEncoder passwordEncoder;
23 |
24 |     @Autowired
25 |     private JwtUtil jwtUtil;
26 |
27 |     @PostMapping("/login")
28 |     public ResponseEntity<?> login(@RequestBody Map<String, String> request) {
29 |         String username = request.get("username");
30 |         String rawPassword = request.get("password");
31 |
32 |         System.out.println("[icono] [LOGIN TEST] Petición recibida para usuario: " + username);
33 |
34 |         Usuario user = usuarioRepository.findByUsername(username).orElse(null);
35 |
36 |         if (user == null) {
37 |             System.out.println("[icono] [LOGIN TEST] Usuario NO encontrado en BD.");
38 |             return ResponseEntity.status(404).body(Map.of("message", "Usuario no encontrado"));
39 |         }
40 |
41 |         System.out.println("[icono] [LOGIN TEST] Usuario encontrado en BD.");
42 |
43 |         boolean coincide = passwordEncoder.matches(rawPassword, user.getPassword());
44 |
45 |         if (!coincide) {
46 |             System.out.println("[icono] [LOGIN TEST] La contraseña NO coincide.");
47 |             return ResponseEntity.status(401).body(Map.of("message", "Credenciales incorrectas"));
```

```
48 |     }
49 |
50 |     System.out.println("[icono] [LOGIN TEST] Autenticación exitosa.");
51 |
52 |     // Genera el token JWT usando el JwtUtil que tienes en el proyecto
53 |     String token = jwtUtil.generateToken(user.getUsername());
54 |
55 |     return ResponseEntity.ok(Map.of(
56 |         "message", "Login exitoso",
57 |         "username", user.getUsername(),
58 |         "token", token
59 |     ));
60 | }
61 | }
```

03. FormularioController.java

src/main/java/com/example/demo/controller/FormularioController.java

104 líneas originales. SHA-256 de revisión: 183e5d7da1a1d22d...

Expone las rutas de personas y contactos. Delega reglas en PersonaService y transforma resultados/excepciones en respuestas HTTP.

Líneas importantes

Línea 13. Prefijo /api/formularios.

Línea 14. Permite el origen localhost:4200 y los métodos/encabezados declarados.

Línea 21. Consulta contactos de una persona identificada por la URL.

Línea 26. Recibe una lista de ContactoDTO para guardarla.

Línea 50. Lista activos; captura RuntimeException como 500 con texto.

Línea 64. Recibe FormularioDTO, devuelve 201 y convierte RuntimeException en 400.

Línea 78. Actualiza por ID; devuelve 200 o 404 ante RuntimeException, incluso cuando es un error de validación.

Línea 92. Baja lógica: 204 sin cuerpo, o 404 si se captura una excepción.

Código numerado

```
1 | package com.example.demo.controller;
2 |
3 | import com.example.demo.dto.FormularioDTO;
4 | import com.example.demo.service.PersonaService;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.http.HttpStatus;
7 | import org.springframework.http.ResponseEntity;
8 | import org.springframework.web.bind.annotation.*;
9 |
10 | import java.util.List;
11 |
12 | @RestController
13 | @RequestMapping("/api/formularios")
14 | @CrossOrigin(
15 |     origins = "http://localhost:4200",
16 |     allowedHeaders = "*",
17 |     methods = {RequestMethod.GET, RequestMethod.POST, RequestMethod.PUT, RequestMethod.DELETE,
18 | RequestMethod.OPTIONS}
19 | )
20 | public class FormularioController {
21 |
22 |     @GetMapping("/{id}/contactos")
23 |     public List<com.example.demo.dto.ContactoDTO> obtenerContactos(@PathVariable Long id) {
24 |         return personaService.obtenerContactos(id);
25 |     }
26 |
27 |     @PutMapping("/{id}/contactos")
28 |     public List<com.example.demo.dto.ContactoDTO> guardarContactos(
29 |         @PathVariable Long id, @RequestBody List<com.example.demo.dto.ContactoDTO> contactos) {
30 |         return personaService.guardarContactos(id, contactos);
31 |     }
32 |
33 |     @Autowired
34 |     private PersonaService personaService;
35 |
36 |     @GetMapping("/inactivos")
37 |     public ResponseEntity<List<FormularioDTO>> obtenerInactivos() {
38 |         return ResponseEntity.ok(personaService.obtenerInactivos());
39 |     }
40 |
41 |     @PutMapping("/{id}/reactivar")
42 |     public ResponseEntity<?> reactivar(@PathVariable Long id) {
43 |         try {
44 |             return ResponseEntity.ok(personaService.reactivar(id));
45 |         } catch (RuntimeException e) {
46 |             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(e.getMessage());
47 |         }
48 |     }
49 | }
```

```

44 |         } catch (org.springframework.web.server.ResponseStatusException e) {
45 |             return ResponseEntity.status(e.getStatusCode()).body(e.getReason());
46 |         }
47 |     }
48 |
49 |     @GetMapping
50 |     public ResponseEntity<?> obtenerFormularios() {
51 |         try {
52 |             List<FormularioDTO> lista = personaService.obtenerTodos();
53 |             return ResponseEntity.ok(lista);
54 |         } catch (org.springframework.web.server.ResponseStatusException e) {
55 |             return ResponseEntity.status(e.getStatusCode()).body(e.getReason());
56 |         } catch (IllegalArgumentException e) {
57 |             return ResponseEntity.badRequest().body(e.getMessage());
58 |         } catch (RuntimeException e) {
59 |             return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(e.getMessage());
60 |         }
61 |     }
62 |
63 |     @PostMapping
64 |     public ResponseEntity<?> guardarFormulario(@RequestBody FormularioDTO dto) {
65 |         try {
66 |             FormularioDTO guardado = personaService.guardar(dto);
67 |             return ResponseEntity.status(HttpStatus.CREATED).body(guardado);
68 |         } catch (org.springframework.web.server.ResponseStatusException e) {
69 |             return ResponseEntity.status(e.getStatusCode()).body(e.getReason());
70 |         } catch (IllegalArgumentException e) {
71 |             return ResponseEntity.badRequest().body(e.getMessage());
72 |         } catch (RuntimeException e) {
73 |             return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
74 |         }
75 |     }
76 |
77 |     @PutMapping("/{id}")
78 |     public ResponseEntity<?> actualizarFormulario(@PathVariable Long id, @RequestBody FormularioDTO
dto) {
79 |         try {
80 |             FormularioDTO actualizado = personaService.actualizar(id, dto);
81 |             return ResponseEntity.ok(actualizado);
82 |         } catch (org.springframework.web.server.ResponseStatusException e) {
83 |             return ResponseEntity.status(e.getStatusCode()).body(e.getReason());
84 |         } catch (IllegalArgumentException e) {
85 |             return ResponseEntity.badRequest().body(e.getMessage());
86 |         } catch (RuntimeException e) {
87 |             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(e.getMessage());
88 |         }
89 |     }
90 |
91 |     @DeleteMapping("/{id}")
92 |     public ResponseEntity<?> eliminarFormulario(@PathVariable Long id) {
93 |         try {
94 |             personaService.eliminarLogico(id);
95 |             return ResponseEntity.noContent().build();
96 |         } catch (org.springframework.web.server.ResponseStatusException e) {
97 |             return ResponseEntity.status(e.getStatusCode()).body(e.getReason());
98 |         } catch (IllegalArgumentException e) {
99 |             return ResponseEntity.badRequest().body(e.getMessage());
100 |         } catch (RuntimeException e) {
101 |             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(e.getMessage());
102 |         }
103 |     }
104 | }

```

04. OcupacionController.java

src/main/java/com/example/demo/controller/OcupacionController.java

20 líneas originales. SHA-256 de revisión: 6ea9621739b41ff0...

Consulta el catálogo directamente a través de su repositorio.

Líneas importantes

Línea 9. Ruta /api/ocupaciones.

Línea 16. Define la consulta GET.

Línea 18. Lee y devuelve todas las ocupaciones con 200.

Código numerado

```
1 | package com.example.demo.controller;
2 |
3 | import com.example.demo.repository.CatalogoOcupacionRepository;
4 | import org.springframework.beans.factory.annotation.Autowired;
5 | import org.springframework.http.ResponseEntity;
6 | import org.springframework.web.bind.annotation.*;
7 |
8 | @RestController
9 | @RequestMapping("/api/ocupaciones")
10 | @CrossOrigin(origins = "http://localhost:4200")
11 | public class OcupacionController {
12 |
13 |     @Autowired
14 |     private CatalogoOcupacionRepository catalogoOcupacionRepository;
15 |
16 |     @GetMapping
17 |     public ResponseEntity<?> obtenerOcupaciones() {
18 |         return ResponseEntity.ok(catalogoOcupacionRepository.findAll());
19 |     }
20 | }
```

05. PersonaService.java

src/main/java/com/example/demo/service/PersonaService.java

343 líneas originales. SHA-256 de revisión: 76fd8b067e9eaa8e...

Registra titulares y relaciona personas de contacto con parentescos. Las operaciones se ejecutan dentro de transacciones.

Líneas importantes

Línea 22. Devuelve todas las relaciones del titular activo.

Línea 27. Sustituye la lista completa; exige al menos uno y no impone máximo.

Línea 51. Lista titulares activos, no personas creadas solo como contacto.

Línea 80. Valida titular, contactos y comunicaciones antes del alta transaccional.

Línea 86. El límite de 20 cuenta solo titulares; no hay bloqueo concurrente.

Línea 91. Identifica y persiste el registro principal como titular.

Línea 92. Persiste las relaciones dentro de la misma transacción.

Línea 98. Actualiza datos/comunicaciones de titular activo; los contactos se gestionan en su propia ruta.

Línea 131. Baja lógica del titular, sin borrar personas ni relaciones compartidas.

Línea 149. Incluye la lista completa en la respuesta, además de los campos singulares del primero.

Línea 164. Devuelve comunicaciones secundarias; la siguiente línea hace lo mismo con teléfonos.

Línea 198. Exige nombre y apellido de hasta 100 caracteres y fecha de nacimiento a los titulares.

Línea 206. Rechaza personas dadas de baja o que solo sean contactos con 404.

Línea 218. Valida mínimo uno, IDs y datos personales requeridos para contactos nuevos.

Línea 247. Busca parentesco existente por ID o nombre exacto; no crea catálogo desde el formulario.

Línea 261. Reutiliza solo personas activas; no modifica datos compartidos.

Línea 265. La persona nueva creada como contacto no cuenta como titular.

Línea 294. Quita relaciones omitidas; orphanRemoval elimina solo los vínculos, nunca personas compartidas.

Línea 297. Devuelve ID de persona, nombre, apellido, teléfono y parentesco de la relación.

Línea 306. Mantiene dos correos y dos teléfonos completos para titulares.

Línea 323. Crea posiciones faltantes y actualiza comunicaciones principales/secundarias.

Código numerado

```
1 | package com.example.demo.service;
2 |
3 | import com.example.demo.dto.FormularioDTO;
4 | import com.example.demo.model.*;
5 | import com.example.demo.repository.CatalogoOcupacionRepository;
6 | import com.example.demo.repository.PersonaRepository;
7 | import org.springframework.beans.factory.annotation.Autowired;
8 | import org.slf4j.Logger;
9 | import org.slf4j.LoggerFactory;
10 | import org.springframework.stereotype.Service;
11 |
12 | import java.time.LocalDate;
13 | import java.util.List;
```

```

14 | import java.util.stream.Collectors;
15 |
16 | @Service
17 | public class PersonaService {
18 |
19 |     private static final Logger LOGGER = LoggerFactory.getLogger(PersonaService.class);
20 |
21 |     @org.springframework.transaction.annotation.Transactional(readOnly = true)
22 |     public List<com.example.demo.dto.ContactoDTO> obtenerContactos(Long id) {
23 |         return titularActivo(id).getContactosEmergencia().stream().map(this::contactoADTO).toList();
24 |     }
25 |
26 |     @org.springframework.transaction.annotation.Transactional
27 |     public List<com.example.demo.dto.ContactoDTO> guardarContactos(Long id,
28 | List<com.example.demo.dto.ContactoDTO> contactos) {
29 |         try {
30 |             validarContactos(contactos);
31 |             Persona titular = titularActivo(id);
32 |             reemplazarContactos(titular, contactos);
33 |             personaRepository.saveAndFlush(titular);
34 |             return titular.getContactosEmergencia().stream().map(this::contactoADTO).toList();
35 |         } catch (RuntimeException exception) {
36 |             LOGGER.error("No fue posible guardar los contactos de la persona {}", id, exception);
37 |             throw exception;
38 |         }
39 |
40 |     @Autowired
41 |     private com.example.demo.repository.CatalogoParentescoRepository parentescoRepository;
42 |
43 |     @Autowired
44 |     private PersonaRepository personaRepository;
45 |
46 |     @Autowired
47 |     private CatalogoOcupacionRepository ocupacionRepository;
48 |
49 |     @org.springframework.transaction.annotation.Transactional(readOnly = true)
50 |     public List<FormularioDTO> obtenerTodos() {
51 |         return
52 | personaRepository.findByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull().stream()
53 |             .map(this::convertirADTO)
54 |             .collect(Collectors.toList());
55 |     }
56 |
57 |     @org.springframework.transaction.annotation.Transactional(readOnly = true)
58 |     public List<FormularioDTO> obtenerInactivos() {
59 |         return
60 | personaRepository.findByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNotNull().stream()
61 |             .map(this::convertirADTO)
62 |             .collect(Collectors.toList());
63 |     }
64 |
65 |     @org.springframework.transaction.annotation.Transactional
66 |     public FormularioDTO reactivar(Long id) {
67 |         Persona persona = personaRepository.findById(id)
68 |             .filter(p -> p.getPerfilTitular() != null && p.getPerfilTitular().getFechaBaja() !=
69 | null)
70 |             .orElseThrow(() -> new org.springframework.web.server.ResponseStatusException(
71 | org.springframework.http.HttpStatus.NOT_FOUND, "Persona desactivada no
72 | encontrada"));
73 |
74 |         if (personaRepository.countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull() >= 20)
75 |         {
76 |             throw new org.springframework.web.server.ResponseStatusException(
77 | org.springframework.http.HttpStatus.CONFLICT, "Se alcanzó el límite de 20
78 | personas activas");
79 |         }
80 |
81 |         persona.getPerfilTitular().setFechaBaja(null);
82 |         return convertirADTO(personaRepository.save(persona));
83 |     }
84 |
85 |     @org.springframework.transaction.annotation.Transactional
86 |     public FormularioDTO guardar(FormularioDTO dto) {
87 |         validarTitular(dto);
88 |         validarContactos(dto.getContactosEmergencia());
89 |         validarComunicacion(dto);
90 |         // Regla de Negocio: Máximo 20 personas activas
91 |         long personasActivas =
92 | personaRepository.countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull();
93 |         if (personasActivas >= 20) {

```

```

87 |         throw new RuntimeException("Límite alcanzado: No se pueden registrar más de 20
| personas activas en el sistema.");
88 |     }
89 |
90 |     Persona persona = convertirEntidad(dto);
91 |     Persona guardada = personaRepository.saveAndFlush(persona);
92 |     reemplazarContactos(guardada, dto.getContactosEmergencia());
93 |     personaRepository.saveAndFlush(guardada);
94 |     return convertirADTO(guardada);
95 | }
96 |
97 | @org.springframework.transaction.annotation.Transactional
98 | public FormularioDTO actualizar(Long id, FormularioDTO dto) {
99 |     validarTitular(dto);
100 |     validarComunicacion(dto);
101 |     Persona persona = titularActivo(id);
102 |
103 |     persona.setNombre(dto.getNombre());
104 |     persona.setApellido(dto.getApellido());
105 |     persona.setFechaNacimiento(dto.getFechaNacimiento());
106 |     persona.setGenero(dto.getGenero());
107 |     PerfilTitular perfil = persona.getPerfilTitular();
108 |     perfil.setDireccion(dto.getDireccion());
109 |     perfil.setCiudad(dto.getCiudad());
110 |
111 |     if (dto.getOcupacion() != null && !dto.getOcupacion().trim().isEmpty()) {
112 |         CatalogoOcupacion ocupacion = ocupacionRepository.findByNombre(dto.getOcupacion())
113 |             .orElseGet(() -> {
114 |                 CatalogoOcupacion nueva = new CatalogoOcupacion();
115 |                 nueva.setNombre(dto.getOcupacion());
116 |                 return ocupacionRepository.save(nueva);
117 |             });
118 |         perfil.setOcupacion(ocupacion);
119 |     }
120 |
121 |
122 |     actualizarComunicacion(persona, dto);
123 |
124 |     Persona actualizada = personaRepository.save(persona);
125 |     return convertirADTO(actualizada);
126 | }
127 |
128 | @org.springframework.transaction.annotation.Transactional
129 | public void eliminarLogico(Long id) {
130 |     Persona persona = titularActivo(id);
131 |     persona.getPerfilTitular().setFechaBaja(LocalDate.now());
132 |     personaRepository.save(persona);
133 | }
134 |
135 | private FormularioDTO convertirADTO(Persona p) {
136 |     FormularioDTO dto = new FormularioDTO();
137 |     dto.setId(p.getId());
138 |     dto.setNombre(p.getNombre());
139 |     dto.setApellido(p.getApellido());
140 |     dto.setFechaNacimiento(p.getFechaNacimiento());
141 |     dto.setGenero(p.getGenero());
142 |     dto.setDireccion(p.getPerfilTitular().getDireccion());
143 |     dto.setCiudad(p.getPerfilTitular().getCiudad());
144 |     dto.setFechaBaja(p.getPerfilTitular().getFechaBaja());
145 |     dto.setOcupacion(p.getPerfilTitular().getOcupacion().getNombre());
146 |
147 |     List<com.example.demo.dto.ContactoDTO> contactos = p.getContactosEmergencia().stream()
148 |         .map(this::contactoADTO).toList();
149 |     dto.setContactosEmergencia(contactos);
150 |     if (!contactos.isEmpty()) {
151 |         dto.setContactoEmergenciaNombre(contactos.get(0).nombre());
152 |         dto.setContactoEmergenciaTelefono(contactos.get(0).telefono());
153 |         dto.setContactoEmergenciaParentesco(contactos.get(0).parentesco());
154 |     }
155 |
156 |     if (p.getCorreos() != null && !p.getCorreos().isEmpty()) {
157 |         dto.setEmail(p.getCorreos().get(0).getCorreo());
158 |     }
159 |
160 |     if (p.getTelefonos() != null && !p.getTelefonos().isEmpty()) {
161 |         dto.setTelefono(p.getTelefonos().get(0).getTelefono());
162 |     }
163 |
164 |
165 |     dto.setCorreosAdicionales(p.getCorreos().stream().skip(1).map(PersonaCorreo::getCorreo).toList());
166 |     dto.setTelefonosAdicionales(p.getTelefonos().stream().skip(1).map(PersonaTelefono::getTelefono).toList());

```

```

no).toList());
166 |         return dto;
167 |     }
168 |
169 |     private Persona convertirAEntidad(FormularioDTO dto) {
170 |         Persona p = new Persona();
171 |         p.setNombre(dto.getNombre());
172 |         p.setApellido(dto.getApellido());
173 |         p.setFechaNacimiento(dto.getFechaNacimiento());
174 |         p.setGenero(dto.getGenero());
175 |
176 |         // Validaci3n expl3cita de ocupaci3n obligatoria
177 |         if (dto.getOcupacion() == null || dto.getOcupacion().trim().isEmpty()) {
178 |             throw new RuntimeException("La ocupaci3n es obligatoria para registrar a la
persona.");
179 |         }
180 |
181 |         CatalogoOcupacion ocupacion = ocupacionRepository.findByNombre(dto.getOcupacion())
182 |             .orElseGet(() -> {
183 |                 CatalogoOcupacion nueva = new CatalogoOcupacion();
184 |                 nueva.setNombre(dto.getOcupacion());
185 |                 return ocupacionRepository.save(nueva);
186 |             });
187 |         PerfilTitular perfil = new PerfilTitular();
188 |         perfil.setDireccion(dto.getDireccion());
189 |         perfil.setCiudad(dto.getCiudad());
190 |         perfil.setOcupacion(ocupacion);
191 |         p.setPerfilTitular(perfil);
192 |
193 |         actualizarComunicacion(p, dto);
194 |
195 |         return p;
196 |     }
197 |
198 |     private void validarTitular(FormularioDTO dto) {
199 |         if (dto == null || dto.getNombre() == null || dto.getNombre().isBlank() ||
dto.getNombre().length() > 100 ||
200 |             dto.getApellido() == null || dto.getApellido().isBlank() ||
dto.getApellido().length() > 100 ||
201 |             dto.getFechaNacimiento() == null) {
202 |             throw invalido("El titular requiere nombre, apellido (hasta 100 caracteres) y fecha de
nacimient");
203 |         }
204 |     }
205 |
206 |     private Persona titularActivo(Long id) {
207 |         return personaRepository.findById(id)
208 |             .filter(p -> p.getPerfilTitular() != null && p.getPerfilTitular().getFechaBaja() ==
null)
209 |             .orElseThrow(() -> new org.springframework.web.server.ResponseStatusException(
210 |                 org.springframework.http.HttpStatus.NOT_FOUND, "Titular no encontrado"));
211 |     }
212 |
213 |     private org.springframework.web.server.ResponseStatusException invalido(String mensaje) {
214 |         return new org.springframework.web.server.ResponseStatusException(
215 |             org.springframework.http.HttpStatus.BAD_REQUEST, mensaje);
216 |     }
217 |
218 |     private void validarContactos(List<com.example.demo.dto.ContactoDTO> contactos) {
219 |         if (contactos == null || contactos.isEmpty()) throw invalido("Se requiere al menos un
contacto de emergencia");
220 |         java.util.Set<Long> ids = new java.util.HashSet<>();
221 |         for (var c : contactos) {
222 |             if (c == null) throw invalido("El contacto no puede ser nulo");
223 |             if (c.getIdContacto() == null &&
224 |                 (c.getNombre() == null || c.getNombre().isBlank() || c.getNombre().length() > 100 ||
225 |                 c.getApellido() == null || c.getApellido().isBlank() || c.getApellido().length() > 100 ||
226 |                 c.getFechaNacimiento() == null || c.getGenero() == null || c.getGenero().isBlank() ||
227 |                 c.getEmail() == null || !c.getEmail().matches("^\\s@]+@[\\s@]+\\.([\\s@]+)$") ||
228 |                 c.getTelefono() == null || !c.getTelefono().matches("[0-9]{10}"))) {
229 |                 throw invalido("El contacto requiere nombre, apellido y telefono valido");
230 |             }
231 |             if (c.getIdContacto() != null) {
232 |                 if (c.getIdContacto() <= 0 || !ids.add(c.getIdContacto()))
233 |                     throw invalido("Los contactos existentes deben tener IDs positivos y no
repetidos");
234 |             } else if (c.getNombre() == null || c.getNombre().isBlank() || c.getNombre().length() > 100 ||
235 |                 c.getApellido() == null || c.getApellido().isBlank() || c.getApellido().length() > 100 ||
236 |                 c.getTelefono() == null || !c.getTelefono().matches("[0-9]{10}")) {
237 |                 throw invalido("El nuevo contacto requiere nombre y apellido de hasta 100 caracteres
y tel3fono de 10 d3gitos");

```

```

238 |         }
239 |         if (c.idParentesco() != null) {
240 |             if (c.idParentesco() <= 0) throw invalido("ID de parentesco invÃ¡lido");
241 |         } else if (c.parentesco() == null || c.parentesco().isBlank() || c.parentesco().length()
> 50) {
242 |             throw invalido("Se requiere un parentesco del catÃ¡logo");
243 |         }
244 |     }
245 | }
246 |
247 | private CatalogoParentesco resolverParentesco(com.example.demo.dto.ContactoDTO dto) {
248 |     return (dto.idParentesco() != null ? parentescoRepository.findById(dto.idParentesco()) :
249 |         parentescoRepository.findByNombre(dto.parentesco().trim()))
250 |         .orElseThrow(() -> invalido("Parentesco inexistente"));
251 | }
252 |
253 | private void reemplazarContactos(Persona titular, List<com.example.demo.dto.ContactoDTO> datos)
{
254 |     var conservadas = new java.util.ArrayList<ContactoEmergencia>();
255 |     for (var dto : datos) {
256 |         if (dto.idContacto() != null && dto.idContacto().equals(titular.getId()))
257 |             throw invalido("Una persona no puede ser su propio contacto");
258 |         CatalogoParentesco parentesco = resolverParentesco(dto);
259 |         Persona contacto;
260 |         if (dto.idContacto() != null) {
261 |             contacto = personaRepository.findById(dto.idContacto())
262 |                 .filter(p -> p.getPerfilTitular() == null ||
p.getPerfilTitular().getFechaBaja() == null)
263 |                 .orElseThrow(() -> invalido("La persona de contacto no existe o estÃ¡ dada
de baja"));
264 |         } else {
265 |             contacto = new Persona();
266 |             contacto.setNombre(dto.nombre().trim());
267 |             contacto.setApellido(dto.apellido().trim());
268 |             contacto.setFechaNacimiento(dto.fechaNacimiento());
269 |             contacto.setGenero(dto.genero().trim());
270 |             PersonaCorreo correo = new PersonaCorreo();
271 |             correo.setPersona(contacto);
272 |             correo.setCorreo(dto.email().trim());
273 |             contacto.getCorreos().add(correo);
274 |             PersonaTelefono telefono = new PersonaTelefono();
275 |             telefono.setPersona(contacto);
276 |             telefono.setTelefono(dto.telefono());
277 |             contacto.getTelefonos().add(telefono);
278 |             contacto = personaRepository.save(contacto);
279 |         }
280 |         Persona personaContacto = contacto;
281 |         ContactoEmergencia relacion = titular.getContactosEmergencia().stream()
282 |             .filter(r -> dto.idContacto() != null &&
dto.idContacto().equals(r.getContacto().getId()))
283 |             .findFirst().orElseGet(() -> {
284 |                 ContactoEmergencia nueva = new ContactoEmergencia();
285 |                 nueva.setPersona(titular);
286 |                 nueva.setContacto(personaContacto);
287 |                 titular.getContactosEmergencia().add(nueva);
288 |                 return nueva;
289 |             });
290 |         relacion.setParentesco(parentesco);
291 |         conservadas.add(relacion);
292 |     }
293 |     // orphanRemoval elimina Ã¼nicamente relaciones, nunca personas compartidas.
294 |     titular.getContactosEmergencia().removeIf(r -> !conservadas.contains(r));
295 | }
296 |
297 | private com.example.demo.dto.ContactoDTO contactoADTO(ContactoEmergencia relacion) {
298 |     Persona contacto = relacion.getContacto();
299 |     String telefono = contacto.getTelefonos().isEmpty() ? null :
contacto.getTelefonos().get(0).getTelefono();
300 |     String email = contacto.getCorreos().isEmpty() ? null :
contacto.getCorreos().get(0).getCorreo();
301 |     return new com.example.demo.dto.ContactoDTO(contacto.getId(), contacto.getNombre(),
contacto.getApellido(),
302 |         contacto.getFechaNacimiento(), contacto.getGenero(), email, telefono,
303 |         relacion.getParentesco().getId(), relacion.getParentesco().getNombre());
304 | }
305 |
306 | private void validarComunicacion(FormularioDTO dto) {
307 |     if (dto.getCorreosAdicionales() == null || dto.getCorreosAdicionales().size() != 1 ||
308 |         dto.getTelefonosAdicionales() == null || dto.getTelefonosAdicionales().size() != 1) {
309 |         throw new IllegalArgumentException("Se requieren un correo y un telÃ©fono
secundarios.");

```

```

310 |     }
311 |     for (String correo : new String[]{dto.getEmail(), dto.getCorreosAdicionales().get(0)}) {
312 |         if (correo == null || correo.length() > 150 ||
313 |         !correo.matches("^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$")) {
314 |             throw new IllegalArgumentException("Los dos correos deben ser válidos.");
315 |         }
316 |     }
317 |     for (String telefono : new String[]{dto.getTelefono(),
318 |     dto.getTelefonosAdicionales().get(0)}) {
319 |         if (telefono == null || !telefono.matches("[0-9]{10}")) {
320 |             throw new IllegalArgumentException("Los dos teléfonos deben tener 10
321 |             dígitos.");
322 |         }
323 |     }
324 | }
325 |
326 | private void actualizarComunicacion(Persona persona, FormularioDTO dto) {
327 |     String[] correos = {dto.getEmail(), dto.getCorreosAdicionales().get(0)};
328 |     String[] telefonos = {dto.getTelefono(), dto.getTelefonosAdicionales().get(0)};
329 |     for (int i = 0; i < 2; i++) {
330 |         if (persona.getCorreos().size() <= i) {
331 |             PersonaCorreo correo = new PersonaCorreo();
332 |             correo.setPersona(persona);
333 |             persona.getCorreos().add(correo);
334 |         }
335 |         persona.getCorreos().get(i).setCorreo(correos[i]);
336 |         if (persona.getTelefonos().size() <= i) {
337 |             PersonaTelefono telefono = new PersonaTelefono();
338 |             telefono.setPersona(persona);
339 |             persona.getTelefonos().add(telefono);
340 |         }
341 |         persona.getTelefonos().get(i).setTelefono(telefonos[i]);
342 |     }
343 | }

```

06. SecurityConfig.java

src/main/java/com/example/demo/config/SecurityConfig.java

54 líneas originales. SHA-256 de revisión: 5a808997dbc45456...

Configura la cadena de seguridad y los beans de autenticación.

Líneas importantes

Línea 18. Habilita esta configuración de seguridad.

Línea 26. Desactiva protección CSRF.

Línea 27. Habilita integración CORS.

Línea 28. No mantiene la autenticación en una sesión HTTP de Spring Security.

Línea 31. Permite OPTIONS.

Línea 32. Permite únicamente el endpoint público de login.

Línea 35. El resto exige identidad autenticada, sin una condición de rol.

Línea 37. Registra JwtFilter en la cadena de Spring Security para establecer la identidad autenticada.

Línea 44. Devuelve un servicio que rechaza consultas; el login se resuelve en el controlador.

Línea 52. Proporciona el comparador/codificador BCrypt.

Código numerado

```
1 | package com.example.demo.config;
2 |
3 | import com.example.demo.security.JwtFilter;
4 | import org.springframework.beans.factory.annotation.Autowired;
5 | import org.springframework.context.annotation.Bean;
6 | import org.springframework.context.annotation.Configuration;
7 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
8 | import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
9 | import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
10 | import org.springframework.security.config.http.SessionCreationPolicy;
11 | import org.springframework.security.core.userdetails.UserDetailsService;
12 | import org.springframework.security.core.userdetails.UsernameNotFoundException;
13 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14 | import org.springframework.security.web.SecurityFilterChain;
15 | import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
16 |
17 | @Configuration
18 | @EnableWebSecurity
19 | public class SecurityConfig {
20 |     @Autowired
21 |     private JwtFilter jwtFilter;
22 |
23 |     @Bean
24 |     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
25 |         http
26 |             .csrf(AbstractHttpConfigurer::disable)
27 |             .cors(cors -> {}) // Funciona si tienes @CrossOrigin en tus controladores
28 |             .sessionManagement(session ->
29 |                 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
30 |             .authorizeHttpRequests(auth -> auth
31 |                 // 1. PERMITIR OPTIONS (Obligatorio para que Angular pueda enviar el Token
32 |                 // sin ser bloqueado)
33 |                 .requestMatchers(org.springframework.http.HttpMethod.OPTIONS,
34 |                 "/**").permitAll()
35 |                 .requestMatchers("/api/auth/login").permitAll()
36 |                 .requestMatchers("/api/formularios/**").authenticated()
37 |                 .requestMatchers("/api/ocupaciones/**").authenticated() // Agrega tus
38 |                 // ocupaciones aquí
39 |                 .anyRequest().authenticated()
40 |             )
41 |             .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
42 |     }
43 | }
```

```

38 |
39 |     return http.build();
40 | }
41 |
42 | // Desactiva la generación de contraseña aleatoria por defecto
43 | @Bean
44 | public UserDetailsServiceImpl userDetailsService() {
45 |     return username -> {
46 |         throw new UsernameNotFoundException("La autenticación es gestionada por
| AuthController");
47 |     };
48 | }
49 |
50 | @Bean
51 | public BCryptPasswordEncoder passwordEncoder() {
52 |     return new BCryptPasswordEncoder();
53 | }
54 | }

```

07. JwtUtil.java

src/main/java/com/example/demo/security/JwtUtil.java

42 líneas originales. SHA-256 de revisión: c196b452076e4841...

Firma y analiza JWT con HS256. No consulta cuentas en la base.

Líneas importantes

Línea 17. Lee jwt.secret desde configuración.

Línea 19. Vigencia fija de 3600000 milisegundos: una hora.

Línea 22. Crea la clave a partir de los bytes UTF-8 del secreto.

Línea 27. Coloca el username como sujeto.

Línea 29. Calcula el vencimiento desde la hora actual.

Línea 30. Firma con HS256.

Línea 38. Analiza y verifica firma y vencimiento; después obtiene el sujeto. Un token incorrecto produce excepción.

Código numerado

```
1 | package com.example.demo.security;
2 |
3 |
4 | import org.springframework.beans.factory.annotation.Value;
5 | import org.springframework.stereotype.Component;
6 | import io.jsonwebtoken.Jwts;
7 | import io.jsonwebtoken.SignatureAlgorithm;
8 | import io.jsonwebtoken.security.Keys;
9 | import java.security.Key;
10 | import java.nio.charset.StandardCharsets;
11 | import java.util.Date;
12 |
13 |
14 | @Component
15 | public class JwtUtil {
16 |
17 |     @Value("${jwt.secret}")
18 |     private String secretKey;
19 |     private final long EXPIRATION_TIME = 3600000; // 1 hora en milisegundos
20 |
21 |     private Key getSigningKey() {
22 |         return Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8));
23 |     }
24 |
25 |     public String generateToken(String username) {
26 |         return Jwts.builder()
27 |             .setSubject(username)
28 |             .setIssuedAt(new Date())
29 |             .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
30 |             .signWith(getSigningKey(), SignatureAlgorithm.HS256)
31 |             .compact();
32 |     }
33 |
34 |     public String extractUsername(String token) {
35 |         return Jwts.parserBuilder()
36 |             .setSigningKey(getSigningKey())
37 |             .build()
38 |             .parseClaimsJws(token)
39 |             .getBody()
40 |             .getSubject();
41 |     }
42 | }
```

08. JwtFilter.java

src/main/java/com/example/demo/security/JwtFilter.java

50 líneas originales. SHA-256 de revisión: 5895680a6e9e649a...

Construye la autenticación que Spring Security utiliza para permitir las rutas protegidas.

Líneas importantes

Línea 17. Registra el filtro como componente administrado por Spring.

Línea 27. Lee Authorization.

Línea 32. Retira el prefijo Bearer y conserva el JWT.

Línea 34. Verifica el token y obtiene el usuario. Los errores se capturan y registran en consola.

Línea 40. Crea autenticación solo si hay sujeto y el contexto aún no tiene una.

Línea 43. Construye autenticación sin autoridades/roles.

Línea 45. Guarda la identidad en el contexto para las siguientes reglas de seguridad.

Línea 48. Continúa la cadena; las reglas posteriores decidirán el acceso cuando no hubo autenticación.

Código numerado

```
1 | package com.example.demo.security;
2 |
3 | import jakarta.servlet.FilterChain;
4 | import jakarta.servlet.ServletException;
5 | import jakarta.servlet.http.HttpServletRequest;
6 | import jakarta.servlet.http.HttpServletResponse;
7 | import org.springframework.beans.factory.annotation.Autowired;
8 | import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
9 | import org.springframework.security.core.context.SecurityContextHolder;
10 | import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
11 | import org.springframework.stereotype.Component;
12 | import org.springframework.web.filter.OncePerRequestFilter;
13 |
14 | import java.io.IOException;
15 | import java.util.Collections;
16 |
17 | @Component
18 | public class JwtFilter extends OncePerRequestFilter {
19 |
20 |     @Autowired
21 |     private JwtUtil jwtUtil;
22 |
23 |     @Override
24 |     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
25 |     FilterChain filterChain)
26 |         throws ServletException, IOException {
27 |
28 |         String authHeader = request.getHeader("Authorization");
29 |         String token = null;
30 |         String username = null;
31 |
32 |         if (authHeader != null && authHeader.startsWith("Bearer ")) {
33 |             token = authHeader.substring(7);
34 |             try {
35 |                 username = jwtUtil.extractUsername(token);
36 |             } catch (Exception e) {
37 |                 System.out.println("[icono][icono] Token JWT inválido o no legible.");
38 |             }
39 |
40 |             if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
41 |                 // Asigna la autenticación dentro del contexto de Spring Security
42 |                 UsernamePasswordAuthenticationToken authToken =
43 |                     new UsernamePasswordAuthenticationToken(username, null,
44 |                     Collections.emptyList());
45 |                 authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
```

```
45 |         SecurityContextHolder.getContext().setAuthentication(authToken);
46 |     }
47 |
48 |     filterChain.doFilter(request, response);
49 | }
50 | }
```

09. FormularioDTO.java

src/main/java/com/example/demo/dto/FormularioDTO.java

65 líneas originales. SHA-256 de revisión: 8b607daa694ac4c1...

Contrato del titular con comunicaciones y lista variable de contactos.

Líneas importantes

Línea 6. Exactamente un correo secundario exigido por el servicio.

Línea 7. Exactamente un teléfono secundario.

Línea 8. Lista de mínimo un contacto, sin máximo; se recibe al crear y se rellena en respuestas.

Línea 17. ID del titular.

Línea 27. Valor no validado ni persistido actualmente.

Línea 28. Campos singulares del primer contacto conservados por compatibilidad.

Línea 35. Accesores para serialización y uso del servicio.

Código numerado

```
1 | package com.example.demo.dto;
2 |
3 | import java.time.LocalDate;
4 |
5 | public class FormularioDTO {
6 |     private java.util.List<String> correosAdicionales = java.util.List.of();
7 |     private java.util.List<String> telefonosAdicionales = java.util.List.of();
8 |     private java.util.List<ContactoDTO> contactosEmergencia;
9 |
10 |     public java.util.List<String> getCorreosAdicionales() { return correosAdicionales; }
11 |     public void setCorreosAdicionales(java.util.List<String> valores) { correosAdicionales =
12 |         valores; }
13 |     public java.util.List<String> getTelefonosAdicionales() { return telefonosAdicionales; }
14 |     public void setTelefonosAdicionales(java.util.List<String> valores) { telefonosAdicionales =
15 |         valores; }
16 |
17 |     private Long id;
18 |     private String nombre;
19 |     private String apellido;
20 |     private String email;
21 |     private String telefono;
22 |     private LocalDate fechaNacimiento;
23 |     private String genero;
24 |     private String direccion;
25 |     private String ciudad;
26 |     private String ocupacion;
27 |     private Boolean aceptaTerminos = true;
28 |     private String contactoEmergenciaNombre;
29 |     private String contactoEmergenciaTelefono;
30 |     private String contactoEmergenciaParentesco;
31 |     private LocalDate fechaBaja;
32 |
33 |     public FormularioDTO() {}
34 |
35 |     public Long getId() { return id; }
36 |     public void setId(Long id) { this.id = id; }
37 |     public String getNombre() { return nombre; }
38 |     public void setNombre(String nombre) { this.nombre = nombre; }
39 |     public String getApellido() { return apellido; }
40 |     public void setApellido(String apellido) { this.apellido = apellido; }
41 |     public String getEmail() { return email; }
42 |     public void setEmail(String email) { this.email = email; }
43 |     public String getTelefono() { return telefono; }
44 |     public void setTelefono(String telefono) { this.telefono = telefono; }
45 |     public LocalDate getFechaNacimiento() { return fechaNacimiento; }
```

```

46 |     public void setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento =
    | fechaNacimiento; }
47 |     public String getGenero() { return genero; }
48 |     public void setGenero(String genero) { this.genero = genero; }
49 |     public String getDireccion() { return direccion; }
50 |     public void setDireccion(String direccion) { this.direccion = direccion; }
51 |     public String getCiudad() { return ciudad; }
52 |     public void setCiudad(String ciudad) { this.ciudad = ciudad; }
53 |     public String getOcupacion() { return ocupacion; }
54 |     public void setOcupacion(String ocupacion) { this.ocupacion = ocupacion; }
55 |     public Boolean getAceptaTerminos() { return aceptaTerminos; }
56 |     public void setAceptaTerminos(Boolean aceptaTerminos) { this.aceptaTerminos = aceptaTerminos; }
57 |     public String getContactoEmergenciaNombre() { return contactoEmergenciaNombre; }
58 |     public void setContactoEmergenciaNombre(String contactoEmergenciaNombre) {
    | this.contactoEmergenciaNombre = contactoEmergenciaNombre; }
59 |     public String getContactoEmergenciaTelefono() { return contactoEmergenciaTelefono; }
60 |     public void setContactoEmergenciaTelefono(String contactoEmergenciaTelefono) {
    | this.contactoEmergenciaTelefono = contactoEmergenciaTelefono; }
61 |     public String getContactoEmergenciaParentesco() { return contactoEmergenciaParentesco; }
62 |     public void setContactoEmergenciaParentesco(String contactoEmergenciaParentesco) {
    | this.contactoEmergenciaParentesco = contactoEmergenciaParentesco; }
63 |     public LocalDate getFechaBaja() { return fechaBaja; }
64 |     public void setFechaBaja(LocalDate fechaBaja) { this.fechaBaja = fechaBaja; }
65 | }

```

10. ContactoDTO.java

src/main/java/com/example/demo/dto/ContactoDTO.java

11 líneas originales. SHA-256 de revisión: d850e945cc3e45f7...

Contrato de una relación de contacto. Los IDs distinguen una persona existente de una nueva.

Líneas importantes

Línea 5. Sin ID se exigen los datos personales requeridos; con ID solo se cambia el vínculo. idParentesco o parentesco identifican el catálogo.

Código numerado

```
1 | package com.example.demo.dto;
2 |
3 | import java.time.LocalDate;
4 |
5 | public record ContactoDTO(Long idContacto, String nombre, String apellido, LocalDate
   | fechaNacimiento,
6 |                               String genero, String email, String telefono, Long idParentesco, String
   | parentesco) {
7 |     /** Compatibilidad temporal para solicitudes antiguas sin datos personales completos. */
8 |     public ContactoDTO(Long idContacto, String nombre, String apellido, String telefono, Long
   | idParentesco, String parentesco) {
9 |         this(idContacto, nombre, apellido, null, null, null, telefono, idParentesco, parentesco);
10 |     }
11 | }
```

11. Persona.java

src/main/java/com/example/demo/model/Persona.java

35 líneas originales. SHA-256 de revisión: 3e962c5906d3b6f2...

Datos compartidos por titulares y contactos. Las colecciones se ordenan por ID ascendente.

Líneas importantes

Línea 9. Mapea la tabla persona.

Línea 14. Fecha obligatoria por esquema.

Línea 28. Indica si existe PerfilTitular; no es una columna propia de Persona.

Línea 25. Accede a los datos exclusivos del registro titular.

Línea 17. Lista relaciones de emergencia sin borrar contactos compartidos.

Línea 17. Colección de vínculos salientes; cascada y eliminación de huérfanos solo sobre relaciones.

Línea 18. Correos ordenados por ID: primero principal y restantes adicionales.

Línea 19. Teléfonos ordenados del mismo modo.

Línea 29. Compatibilidad: titular se deriva de la existencia de PerfilTitular.

Línea 32. Devuelve relaciones salientes ordenadas.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDate;
5 | import java.util.ArrayList;
6 | import java.util.List;
7 |
8 | @Entity
9 | @Table(name = "persona")
10 | public class Persona {
11 |     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12 |     @Column(nullable = false, length = 100) private String nombre;
13 |     @Column(nullable = false, length = 100) private String apellido;
14 |     @Column(name = "fecha_nacimiento", nullable = false) private LocalDate fechaNacimiento;
15 |     @Column(nullable = false, length = 50) private String genero;
16 |     @OneToOne(mappedBy = "persona", cascade = CascadeType.ALL, orphanRemoval = true) private
17 |     PerfilTitular perfilTitular;
18 |     @OneToMany(mappedBy = "persona", cascade = CascadeType.ALL, orphanRemoval = true) @OrderBy("id
19 |     ASC") private List<ContactoEmergencia> contactosEmergencia = new ArrayList<>();
20 |     @OneToMany(mappedBy = "persona", cascade = CascadeType.ALL, orphanRemoval = true) @OrderBy("id
21 |     ASC") private List<PersonaCorreo> correos = new ArrayList<>();
22 |     @OneToMany(mappedBy = "persona", cascade = CascadeType.ALL, orphanRemoval = true) @OrderBy("id
23 |     ASC") private List<PersonaTelefono> telefonos = new ArrayList<>();
24 |     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
25 |     public String getNombre() { return nombre; } public void setNombre(String nombre) { this.nombre
26 |     = nombre; }
27 |     public String getApellido() { return apellido; } public void setApellido(String apellido) {
28 |     this.apellido = apellido; }
29 |     public LocalDate getFechaNacimiento() { return fechaNacimiento; } public void
30 |     setFechaNacimiento(LocalDate fechaNacimiento) { this.fechaNacimiento = fechaNacimiento; }
31 |     public String getGenero() { return genero; } public void setGenero(String genero) { this.genero
32 |     = genero; }
33 |     public PerfilTitular getPerfilTitular() { return perfilTitular; }
34 |     public void setPerfilTitular(PerfilTitular perfilTitular) { this.perfilTitular = perfilTitular;
35 |     if (perfilTitular != null) perfilTitular.setPersona(this); }
36 |     /** Compatibilidad para código que todavía consulta si una persona es titular. */
37 |     public boolean isTitular() { return perfilTitular != null; }
38 |     public void setTitular(boolean titular) { if (titular && perfilTitular == null)
39 |     setPerfilTitular(new PerfilTitular()); if (!titular) perfilTitular = null; }
40 |     public java.time.LocalDate getFechaBaja() { return perfilTitular == null ? null :
41 |     perfilTitular.getFechaBaja(); }
```

```
31 |     public void setFechaBaja(java.time.LocalDate fechaBaja) { if (perfilTitular == null)
    | setPerfilTitular(new PerfilTitular()); perfilTitular.setFechaBaja(fechaBaja); }
32 |     public List<ContactoEmergencia> getContactosEmergencia() { return contactosEmergencia; }
33 |     public List<PersonaCorreo> getCorreos() { return correos; }
34 |     public List<PersonaTelefono> getTelefonos() { return telefonos; }
35 | }
```

12. PerfilTitular.java

src/main/java/com/example/demo/model/PerfilTitular.java

20 líneas originales. SHA-256 de revisión: a43ebb58c106f744...

Datos exclusivos del perfil de una persona registrada como titular.

Líneas importantes

Línea 10. Comparte la clave primaria con Persona.

Línea 13. Requiere una ocupación del catálogo.

Línea 14. null indica perfil activo; una fecha registra la baja lógica.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDate;
5 |
6 | @Entity
7 | @Table(name = "perfil_titular")
8 | public class PerfilTitular {
9 |     @Id @Column(name = "id_persona") private Long id;
10 |     @OneToOne(optional = false) @MapsId @JoinColumn(name = "id_persona") private Persona persona;
11 |     @Column(nullable = false, length = 255) private String direccion;
12 |     @Column(nullable = false, length = 100) private String ciudad;
13 |     @ManyToOne(optional = false) @JoinColumn(name = "id_ocupacion") private CatalogoOcupacion
    | ocupacion;
14 |     @Column(name = "fecha_baja") private LocalDate fechaBaja;
15 |     public Persona getPersona() { return persona; } public void setPersona(Persona persona) {
    | this.persona = persona; }
16 |     public String getDireccion() { return direccion; } public void setDireccion(String direccion) {
    | this.direccion = direccion; }
17 |     public String getCiudad() { return ciudad; } public void setCiudad(String ciudad) { this.ciudad
    | = ciudad; }
18 |     public CatalogoOcupacion getOcupacion() { return ocupacion; } public void
    | setOcupacion(CatalogoOcupacion ocupacion) { this.ocupacion = ocupacion; }
19 |     public LocalDate getFechaBaja() { return fechaBaja; } public void setFechaBaja(LocalDate
    | fechaBaja) { this.fechaBaja = fechaBaja; }
20 | }
```

13. CatalogoOcupacion.java

src/main/java/com/example/demo/model/CatalogoOcupacion.java

21 líneas originales. SHA-256 de revisión: 81a6d1ff108a1d5f...

Nombre de ocupación reutilizado por personas.

Líneas importantes

Línea 6. Identifica la tabla SQL.

Línea 9. IDENTITY delega la generación de ID en la base.

Línea 13. Campo persistido; las anotaciones Column cercanas describen restricciones del mapeo.

Línea 17. Inicio de accesores para leer/escribir campos; no contienen validación de negocio.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "catalogo_ocupacion")
7 | public class CatalogoOcupacion {
8 |     @Id
9 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
10 |     private Long id;
11 |
12 |     @Column(nullable = false, length = 100)
13 |     private String nombre;
14 |
15 |     public CatalogoOcupacion() {}
16 |
17 |     public Long getId() { return id; }
18 |     public void setId(Long id) { this.id = id; }
19 |     public String getNombre() { return nombre; }
20 |     public void setNombre(String nombre) { this.nombre = nombre; }
21 | }
```

14. PersonaCorreo.java

src/main/java/com/example/demo/model/PersonaCorreo.java

27 líneas originales. SHA-256 de revisión: 130ff10cee64f187...

Correo asociado a una persona; longitud alineada en 150 con SQL y servicio.

Líneas importantes

Línea 6. Mapea persona_correo.

Línea 9. Identificador generado por base.

Línea 13. Vincula al padre por id_persona.

Línea 16. Longitud y obligatoriedad del correo.

Línea 25. Acceso al valor para construir el DTO.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "persona_correo")
7 | public class PersonaCorreo {
8 |     @Id
9 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
10 |    private Long id;
11 |
12 |    @ManyToOne
13 |    @JoinColumn(name = "id_persona", nullable = false)
14 |    private Persona persona;
15 |
16 |    @Column(nullable = false, length = 150)
17 |    private String correo;
18 |
19 |    public PersonaCorreo() {}
20 |
21 |    public Long getId() { return id; }
22 |    public void setId(Long id) { this.id = id; }
23 |    public Persona getPersona() { return persona; }
24 |    public void setPersona(Persona persona) { this.persona = persona; }
25 |    public String getCorreo() { return correo; }
26 |    public void setCorreo(String correo) { this.correo = correo; }
27 | }
```

15. PersonaTelefono.java

src/main/java/com/example/demo/model/PersonaTelefono.java

27 líneas originales. SHA-256 de revisión: 351ff611dad1df48...

Teléfono asociado a una persona; SQL/JPA admiten 30 caracteres y el alta HTTP exige diez dígitos.

Líneas importantes

Línea 6. Mapea persona_telefono.

Línea 9. Identificador generado por base.

Línea 13. Relación ManyToOne a persona.

Línea 16. Longitud alineada con schema.sql.

Línea 25. Devuelve el número.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "persona_telefono")
7 | public class PersonaTelefono {
8 |     @Id
9 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
10 |    private Long id;
11 |
12 |    @ManyToOne
13 |    @JoinColumn(name = "id_persona", nullable = false)
14 |    private Persona persona;
15 |
16 |    @Column(nullable = false, length = 30)
17 |    private String telefono;
18 |
19 |    public PersonaTelefono() {}
20 |
21 |    public Long getId() { return id; }
22 |    public void setId(Long id) { this.id = id; }
23 |    public Persona getPersona() { return persona; }
24 |    public void setPersona(Persona persona) { this.persona = persona; }
25 |    public String getTelefono() { return telefono; }
26 |    public void setTelefono(String telefono) { this.telefono = telefono; }
27 | }
```

16. ContactoEmergencia.java

src/main/java/com/example/demo/model/ContactoEmergencia.java

36 líneas originales. SHA-256 de revisión: fcd82530e62f2f18...

Entidad de unión entre dos personas, con parentesco propio de cada vínculo. No contiene nombre o teléfono duplicados.

Líneas importantes

Línea 8. Mapea persona_contacto_emergencia; UNIQUE evita repetir el par titular/contacto.

Línea 7. Impide autorreferencia también al generar el esquema JPA.

Línea 16. Referencia al titular.

Línea 21. Referencia a la persona compartida, sin cascada de borrado.

Línea 25. Referencia al catálogo de parentescos.

Línea 32. Acceso a los datos de la persona de contacto.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | /** Relación dirigida titular -> persona de contacto, con parentesco propio. */
6 | @Entity
7 | @org.hibernate.annotations.Check(constraints = "id_persona <> id_contacto")
8 | @Table(name = "persona_contacto_emergencia",
9 |       uniqueConstraints = @UniqueConstraint(name = "uq_pce_par", columnNames = {"id_persona",
10 |      "id_contacto"}))
11 | public class ContactoEmergencia {
12 |     @Id
13 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
14 |     private Long id;
15 |
16 |     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17 |     @JoinColumn(name = "id_persona", nullable = false)
18 |     private Persona persona;
19 |
20 |     // Sin cascada REMOVE: el contacto puede estar compartido con otros titulares.
21 |     @ManyToOne(fetch = FetchType.LAZY, optional = false)
22 |     @JoinColumn(name = "id_contacto", nullable = false)
23 |     private Persona contacto;
24 |
25 |     @ManyToOne(fetch = FetchType.LAZY, optional = false)
26 |     @JoinColumn(name = "id_parentesco", nullable = false)
27 |     private CatalogoParentesco parentesco;
28 |
29 |     public Long getId() { return id; }
30 |     public void setId(Long id) { this.id = id; }
31 |     public Persona getPersona() { return persona; }
32 |     public void setPersona(Persona persona) { this.persona = persona; }
33 |     public Persona getContacto() { return contacto; }
34 |     public void setContacto(Persona contacto) { this.contacto = contacto; }
35 |     public CatalogoParentesco getParentesco() { return parentesco; }
36 |     public void setParentesco(CatalogoParentesco parentesco) { this.parentesco = parentesco; }
37 | }
```

17. Usuario.java

src/main/java/com/example/demo/model/Usuario.java

33 líneas originales. SHA-256 de revisión: d491ac5da4b51d12...

Cuenta con username, hash y rol. No mapea id_persona aunque el script SQL lo declara obligatorio/único.

Líneas importantes

Línea 6. Identifica la tabla SQL.

Línea 10. IDENTITY delega la generación de ID en la base.

Línea 17. Campo persistido; las anotaciones Column cercanas describen restricciones del mapeo.

Línea 19. Guarda el rol pero el login/filtros actuales no lo verifican.

Línea 22. Inicio de accesores para leer/escribir campos; no contienen validación de negocio.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "usuario")
7 | public class Usuario {
8 |
9 |     @Id
10 |    @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |    private Long id;
12 |
13 |    @Column(unique = true, nullable = false)
14 |    private String username;
15 |
16 |    @Column(nullable = false)
17 |    private String password;
18 |
19 |    private String rol;
20 |
21 |    // Getters y Setters
22 |    public Long getId() { return id; }
23 |    public void setId(Long id) { this.id = id; }
24 |
25 |    public String getUsername() { return username; }
26 |    public void setUsername(String username) { this.username = username; }
27 |
28 |    public String getPassword() { return password; }
29 |    public void setPassword(String password) { this.password = password; }
30 |
31 |    public String getRol() { return rol; }
32 |    public void setRol(String rol) { this.rol = rol; }
33 | }
```

18. PersonaRepository.java

src/main/java/com/example/demo/repository/PersonaRepository.java

13 líneas originales. SHA-256 de revisión: 3a8f22149ce61851...

Acceso a personas con consultas explícitas para titulares activos.

Líneas importantes

Línea 9. Hereda persistencia y búsqueda por ID para titulares y contactos.

Línea 10. Lista solo titulares activos.

Línea 11. Lista titulares dados de baja.

Línea 12. Cuenta solo titulares activos, excluyendo contactos del límite de 20.

Código numerado

```
1 | package com.example.demo.repository;
2 |
3 | import com.example.demo.model.Persona;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 | import java.util.List;
7 |
8 | @Repository
9 | public interface PersonaRepository extends JpaRepository<Persona, Long> {
10 |     List<Persona> findByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull();
11 |     List<Persona> findByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNotNull();
12 |     long countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull();
13 | }
```

19. CatalogoOcupacionRepository.java

src/main/java/com/example/demo/repository/CatalogoOcupacionRepository.java

11 líneas originales. SHA-256 de revisión: fb52acec4231c8b...

Consultas y persistencia de ocupaciones.

Líneas importantes

Línea 9. Hereda save, saveAndFlush, findById, findAll y otras operaciones; el ID es Long.

Línea 10. Spring Data deriva la consulta del nombre del método.

Línea 10. Expresa que la búsqueda puede no encontrar coincidencia.

Código numerado

```
1 | package com.example.demo.repository;
2 |
3 | import com.example.demo.model.CatalogoOcupacion;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 | import java.util.Optional;
7 |
8 | @Repository
9 | public interface CatalogoOcupacionRepository extends JpaRepository<CatalogoOcupacion, Long> {
10 |     Optional<CatalogoOcupacion> findByNombre(String nombre);
11 | }
```

20. UsuarioRepository.java

src/main/java/com/example/demo/repository/UsuarioRepository.java

9 líneas originales. SHA-256 de revisión: 7b2f46fe169254d4...

Consultas y persistencia de cuentas.

Líneas importantes

Línea 7. Hereda save, saveAndFlush, findById, findAll y otras operaciones; el ID es Long.

Línea 8. Spring Data deriva la consulta del nombre del método.

Línea 8. Expresa que la búsqueda puede no encontrar coincidencia.

Código numerado

```
1 | package com.example.demo.repository;
2 |
3 | import com.example.demo.model.Usuario;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import java.util.Optional;
6 |
7 | public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
8 |     Optional<Usuario> findByUsername(String username);
9 | }
```

21. application.properties

src/main/resources/application.properties

11 líneas originales. SHA-256 de revisión: e554a6d068c355f0...

Configuración de conexión, clave JWT e Hibernate. Los secretos se omiten en el PDF sin alterar números de línea.

Líneas importantes

Línea 1. Nombre de aplicación.

Línea 2. Conexión PostgreSQL a nomina_db.

Línea 3. Usuario de la conexión.

Línea 4. Contraseña configurada directamente; valor omitido en esta copia.

Línea 6. Clave de firma configurada directamente; valor omitido.

Línea 9. update solicita ajustar el esquema JPA; no equivale a ejecutar data.sql.

Línea 10. Muestra consultas SQL.

Línea 11. Dialecto PostgreSQL. No aparece habilitación de inicialización SQL automática.

Código numerado

```
1 | spring.application.name=demo
2 | spring.datasource.url=jdbc:postgresql://localhost:5432/nomina_db
3 | spring.datasource.username=postgres
4 | spring.datasource.password=<VALOR OMITIDO>
5 | spring.datasource.driver-class-name=org.postgresql.Driver
6 | jwt.secret=<VALOR OMITIDO>
7 |
8 |
9 | spring.jpa.hibernate.ddl-auto=update
10 | spring.jpa.show-sql=true
11 | spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

22. schema.sql

src/main/resources/schema.sql

49 líneas originales. SHA-256 de revisión: a89f84dbc620f573...

Esquema con contactos como personas y unión N a N con parentesco.

Líneas importantes

Línea 2. Catálogo de parentescos.

Línea 5. Datos personales comunes y obligatorios de titulares y contactos.

Línea 14. Datos complementarios y fecha de baja lógica de titulares.

Línea 34. Unión entre titular y contacto con parentesco.

Línea 39. Restricción que evita repetir el par.

Línea 40. Prohíbe una persona como contacto de sí misma.

Línea 42. Facilita búsqueda de relaciones salientes.

Línea 22. Correos de hasta 150 caracteres.

Línea 28. Teléfonos de exactamente diez dígitos.

Línea 43. Cuenta con id_persona obligatorio, todavía no mapeado en la entidad Usuario.

Código numerado

```
1 | CREATE TABLE IF NOT EXISTS catalogo_ocupacion (id BIGSERIAL PRIMARY KEY, nombre VARCHAR(100) NOT
  | NULL UNIQUE);
2 | CREATE TABLE IF NOT EXISTS catalogo_parentesco (id BIGSERIAL PRIMARY KEY, nombre VARCHAR(50) NOT
  | NULL UNIQUE);
3 |
4 | -- Datos compartidos por titulares, contactos y administradores.
5 | CREATE TABLE IF NOT EXISTS persona (
6 | id BIGSERIAL PRIMARY KEY,
7 | nombre VARCHAR(100) NOT NULL,
8 | apellido VARCHAR(100) NOT NULL,
9 | fecha_nacimiento DATE NOT NULL,
10 | genero VARCHAR(50) NOT NULL
11 | );
12 |
13 | -- Solo existe para personas registradas como titulares.
14 | CREATE TABLE IF NOT EXISTS perfil_titular (
15 | id_persona BIGINT PRIMARY KEY REFERENCES persona(id) ON DELETE CASCADE,
16 | direccion VARCHAR(255) NOT NULL,
17 | ciudad VARCHAR(100) NOT NULL,
18 | id_ocupacion BIGINT NOT NULL REFERENCES catalogo_ocupacion(id),
19 | fecha_baja DATE
20 | );
21 |
22 | CREATE TABLE IF NOT EXISTS persona_correo (
23 | id BIGSERIAL PRIMARY KEY,
24 | id_persona BIGINT NOT NULL REFERENCES persona(id) ON DELETE CASCADE,
25 | correo VARCHAR(150) NOT NULL,
26 | CONSTRAINT uq_persona_correo UNIQUE (id_persona, correo)
27 | );
28 | CREATE TABLE IF NOT EXISTS persona_telefono (
29 | id BIGSERIAL PRIMARY KEY,
30 | id_persona BIGINT NOT NULL REFERENCES persona(id) ON DELETE CASCADE,
31 | telefono VARCHAR(10) NOT NULL CHECK (telefono ~ '^[0-9]{10}$'),
32 | CONSTRAINT uq_persona_telefono UNIQUE (id_persona, telefono)
33 | );
34 | CREATE TABLE IF NOT EXISTS persona_contacto_emergencia (
35 | id BIGSERIAL PRIMARY KEY,
36 | id_persona BIGINT NOT NULL REFERENCES persona(id) ON DELETE CASCADE,
37 | id_contacto BIGINT NOT NULL REFERENCES persona(id) ON DELETE CASCADE,
38 | id_parentesco BIGINT NOT NULL REFERENCES catalogo_parentesco(id),
39 | CONSTRAINT uq_pce_par UNIQUE (id_persona, id_contacto),
```

```
40 | CONSTRAINT ck_pce_distintos CHECK (id_persona <> id_contacto)
41 | );
42 | CREATE INDEX IF NOT EXISTS idx_pce_persona ON persona_contacto_emergencia(id_persona);
43 | CREATE TABLE IF NOT EXISTS usuario (
44 | id BIGSERIAL PRIMARY KEY,
45 | username VARCHAR(50) NOT NULL UNIQUE,
46 | password VARCHAR(255) NOT NULL,
47 | rol VARCHAR(20) NOT NULL DEFAULT 'ROLE_ADMIN',
48 | id_persona BIGINT NOT NULL UNIQUE REFERENCES persona(id) ON DELETE CASCADE
49 | );
```

23. data.sql

src/main/resources/data.sql

39 líneas originales. SHA-256 de revisión: 1e4c1d36b2bc1618...

Semilla con ocupaciones, parentescos, dos titulares, dos contactos-persona, comunicaciones y cuentas.

Líneas importantes

Línea 1. Siete ocupaciones iniciales.

Línea 5. Once parentescos iniciales.

Línea 11. Crea titulares y contactos con datos personales completos.

Línea 19. Distingue los dos titulares iniciales y sus datos complementarios.

Línea 22. Correos iniciales; repetir la semilla puede duplicarlos.

Línea 28. Teléfonos de titulares y contactos.

Línea 32. Vínculos titular/contacto/parentesco; evita duplicar el mismo par.

Línea 35. Cuentas con hashes BCrypt; no se verificaron contraseñas.

Línea 39. Ajusta la secuencia de personas después de IDs explícitos.

Código numerado

```
1 | INSERT INTO catalogo_ocupacion (id, nombre) VALUES
2 | (1, 'Ingeniero de Software'), (2, 'Administrador del Sistema'), (3, 'Medico Cirujano'),
3 | (4, 'Diseñador Grafico'), (5, 'Contador Publico'), (6, 'Docente Universitario'), (7, 'Abogado')
4 | ON CONFLICT (id) DO NOTHING;
5 | INSERT INTO catalogo_parentesco (id, nombre) VALUES
6 | (1, 'Padre'), (2, 'Madre'), (3, 'Hermano(a)'), (4, 'Conyuge'), (5, 'Hijo(a)'),
7 | (6, 'Abuelo(a)'), (7, 'Tio(a)'), (8, 'Primo(a)'), (9, 'Amigo(a)'), (10, 'Empresa'), (11, 'Otro')
8 | ON CONFLICT (id) DO NOTHING;
9 |
10 | -- Administradores y contactos: toda persona inicia con datos personales completos.
11 | INSERT INTO persona (id, nombre, apellido, fecha_nacimiento, genero) VALUES
12 | (1, 'Carlos', 'Gomez', '1990-03-15', 'Masculino'),
13 | (2, 'Laura', 'Martinez', '1993-08-22', 'Femenino'),
14 | (3, 'Andrea', 'Hernandez', '1968-11-02', 'Femenino'),
15 | (4, 'Miguel', 'Santos', '1985-06-19', 'Masculino'),
16 | (5, 'Sofia', 'Ramirez', '1975-02-27', 'Femenino'),
17 | (6, 'Daniel', 'Ortega', '1970-09-12', 'Masculino')
18 | ON CONFLICT (id) DO NOTHING;
19 | INSERT INTO perfil_titular (id_persona, direccion, ciudad, id_ocupacion, fecha_baja) VALUES
20 | (1, 'Av. Revolucion 101', 'Pachuca', 2, NULL), (2, 'Calle Allende 202', 'Pachuca', 2, NULL)
21 | ON CONFLICT (id_persona) DO NOTHING;
22 | INSERT INTO persona_correo (id_persona, correo) VALUES
23 | (1, 'carlos.admin@sistema.com'), (1, 'carlos.soporte@sistema.com'),
24 | (2, 'laura.admin@sistema.com'), (2, 'laura.soporte@sistema.com'),
25 | (3, 'andrea.hernandez@email.com'), (4, 'miguel.santos@email.com'),
26 | (5, 'sofia.ramirez@email.com'), (6, 'daniel.ortega@email.com')
27 | ON CONFLICT DO NOTHING;
28 | INSERT INTO persona_telefono (id_persona, telefono) VALUES
29 | (1, '7711234567'), (1, '7711234568'), (2, '7719876543'), (2, '7719876544'),
30 | (3, '7714000001'), (4, '7714000002'), (5, '7714000003'), (6, '7714000004')
31 | ON CONFLICT DO NOTHING;
32 | INSERT INTO persona_contacto_emergencia (id_persona, id_contacto, id_parentesco) VALUES
33 | (1, 3, 2), (1, 4, 9), (2, 5, 3), (2, 6, 1)
34 | ON CONFLICT DO NOTHING;
35 | INSERT INTO usuario (id, username, password, rol, id_persona) VALUES
36 | (1, 'admin1', '<HASH OMITIDO>', 'ROLE_ADMIN', 1),
37 | (2, 'admin2', '<HASH OMITIDO>', 'ROLE_ADMIN', 2)
38 | ON CONFLICT (id) DO NOTHING;
39 | SELECT setval('persona_id_seq', (SELECT MAX(id) FROM persona));
```

24. pom.xml

pom.xml

137 líneas originales. SHA-256 de revisión: 08d655df6a9f0862...

Define versiones, dependencias y tareas de compilación y documentación con Maven.

Líneas importantes

Línea 7. Hereda configuración de Spring Boot; la siguiente línea fija la versión 4.1.1.

Línea 30. Java 21.

Línea 43. Soporte de controladores HTTP.

Línea 73. Driver de base de datos en ejecución.

Línea 78. Persistencia JPA y repositorios.

Línea 82. API JWT; después aparecen implementación y Jackson.

Línea 99. Dependencia de seguridad.

Línea 107. Procesa documentación en prepare-package.

Línea 132. Empaquetado y ejecución de la aplicación Spring Boot.

Código numerado

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
   | instance"
3 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
   | https://maven.apache.org/xsd/maven-4.0.0.xsd">
4 |     <modelVersion>4.0.0</modelVersion>
5 |     <parent>
6 |         <groupId>org.springframework.boot</groupId>
7 |         <artifactId>spring-boot-starter-parent</artifactId>
8 |         <version>4.1.1</version>
9 |         <relativePath/> <!-- lookup parent from repository -->
10 |     </parent>
11 |     <groupId>com.example</groupId>
12 |     <artifactId>demo</artifactId>
13 |     <version>0.0.1-SNAPSHOT</version>
14 |     <name/>
15 |     <description/>
16 |     <url/>
17 |     <licenses>
18 |         <license/>
19 |     </licenses>
20 |     <developers>
21 |         <developer/>
22 |     </developers>
23 |     <scm>
24 |         <connection/>
25 |         <developerConnection/>
26 |         <tag/>
27 |         <url/>
28 |     </scm>
29 |     <properties>
30 |         <java.version>21</java.version>
31 |     </properties>
32 |     <dependencies>
33 |         <dependency>
34 |             <groupId>org.springframework.boot</groupId>
35 |             <artifactId>spring-boot-starter-jdbc</artifactId>
36 |         </dependency>
37 |         <dependency>
38 |             <groupId>org.springframework.boot</groupId>
39 |             <artifactId>spring-boot-starter-restclient</artifactId>
40 |         </dependency>
41 |         <dependency>
42 |             <groupId>org.springframework.boot</groupId>
```

```

43 |         <artifactId>spring-boot-starter-webmvc</artifactId>
44 |     </dependency>
45 |
46 |     <dependency>
47 |         <groupId>org.springframework.boot</groupId>
48 |         <artifactId>spring-boot-starter-jdbc-test</artifactId>
49 |         <scope>test</scope>
50 |     </dependency>
51 |     <dependency>
52 |         <groupId>org.springframework.boot</groupId>
53 |         <artifactId>spring-boot-starter-restclient-test</artifactId>
54 |         <scope>test</scope>
55 |     </dependency>
56 |     <dependency>
57 |         <groupId>org.springframework.boot</groupId>
58 |         <artifactId>spring-boot-starter-restdocs</artifactId>
59 |         <scope>test</scope>
60 |     </dependency>
61 |     <dependency>
62 |         <groupId>org.springframework.boot</groupId>
63 |         <artifactId>spring-boot-starter-webmvc-test</artifactId>
64 |         <scope>test</scope>
65 |     </dependency>
66 |     <dependency>
67 |         <groupId>org.springframework.restdocs</groupId>
68 |         <artifactId>spring-restdocs-mockmvc</artifactId>
69 |         <scope>test</scope>
70 |     </dependency>
71 |     <dependency>
72 |         <groupId>org.postgresql</groupId>
73 |         <artifactId>postgresql</artifactId>
74 |         <scope>runtime</scope>
75 |     </dependency>
76 |     <dependency>
77 |         <groupId>org.springframework.boot</groupId>
78 |         <artifactId>spring-boot-starter-data-jpa</artifactId>
79 |     </dependency>
80 |     <dependency>
81 |         <groupId>io.jsonwebtoken</groupId>
82 |         <artifactId>jjwt-api</artifactId>
83 |         <version>0.11.5</version>
84 |     </dependency>
85 |     <dependency>
86 |         <groupId>io.jsonwebtoken</groupId>
87 |         <artifactId>jjwt-impl</artifactId>
88 |         <version>0.11.5</version>
89 |         <scope>runtime</scope>
90 |     </dependency>
91 |     <dependency>
92 |         <groupId>io.jsonwebtoken</groupId>
93 |         <artifactId>jjwt-jackson</artifactId>
94 |         <version>0.11.5</version>
95 |         <scope>runtime</scope>
96 |     </dependency>
97 |     <dependency>
98 |         <groupId>org.springframework.boot</groupId>
99 |         <artifactId>spring-boot-starter-security</artifactId>
100 |     </dependency>
101 | </dependencies>
102 |
103 | <build>
104 |     <plugins>
105 |         <plugin>
106 |             <groupId>org.asciidoctor</groupId>
107 |             <artifactId>asciidoctor-maven-plugin</artifactId>
108 |             <version>2.2.1</version>
109 |             <executions>
110 |                 <execution>
111 |                     <id>generate-docs</id>
112 |                     <phase>prepare-package</phase>
113 |                     <goals>
114 |                         <goal>process-asciidoc</goal>
115 |                     </goals>
116 |                     <configuration>
117 |                         <backend>html</backend>
118 |                         <doctype>book</doctype>
119 |                     </configuration>
120 |                 </execution>
121 |             </executions>
122 |             <dependencies>
123 |                 <dependency>

```

```

124 |             <groupId>org.springframework.restdocs</groupId>
125 |             <artifactId>spring-restdocs-asciidoctor</artifactId>
126 |             <version>${spring-restdocs.version}</version>
127 |         </dependency>
128 |     </dependencies>
129 | </plugin>
130 | <plugin>
131 |     <groupId>org.springframework.boot</groupId>
132 |     <artifactId>spring-boot-maven-plugin</artifactId>
133 | </plugin>
134 | </plugins>
135 | </build>
136 |
137 | </project>

```

25. HashPassword.java

scripts/HashPassword.java

30 líneas originales. SHA-256 de revisión: d0f81902d234fd2b...

Utilidad interactiva para crear un hash y guardarlo en usuario.password. No actualiza la base por sí sola.

Líneas importantes

Línea 10. Exige una consola interactiva.

Línea 14. Lee contraseña sin mostrarla; después solicita confirmación.

Línea 17. Rechaza valores ausentes o diferentes.

Línea 21. Exige mínimo 12 caracteres y máximo 72 bytes UTF-8.

Línea 24. Genera e imprime BCrypt con coste 12.

Línea 26. Limpia arrays en finally; no borra el String creado previamente.

Código numerado

```
1 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
2 |
3 | import java.io.Console;
4 | import java.nio.charset.StandardCharsets;
5 | import java.util.Arrays;
6 |
7 | /** Ejecutar desde una terminal con el classpath generado por Maven; ver README.md. */
8 | class HashPassword {
9 |     public static void main(String[] args) {
10 |         Console console = System.console();
11 |         if (console == null) {
12 |             throw new IllegalStateException("Ejecuta esta utilidad desde una terminal interactiva");
13 |         }
14 |         char[] password = console.readPassword("Nueva contraseña: ");
15 |         char[] confirmation = console.readPassword("Confirmar contraseña: ");
16 |         try {
17 |             if (password == null || confirmation == null || !Arrays.equals(password, confirmation))
18 |             {
19 |                 throw new IllegalArgumentException("Las contraseñas no coinciden");
20 |             }
21 |             String value = new String(password);
22 |             if (value.isBlank() || value.length() < 12 ||
23 |                 value.getBytes(StandardCharsets.UTF_8).length > 72) {
24 |                 throw new IllegalArgumentException("Usa al menos 12 caracteres y como máximo 72
25 |                 bytes UTF-8");
26 |             }
27 |             System.out.println(new BCryptPasswordEncoder(12).encode(value));
28 |         } finally {
29 |             if (password != null) Arrays.fill(password, '\0');
30 |             if (confirmation != null) Arrays.fill(confirmation, '\0');
31 |         }
32 |     }
33 | }
```

26. RegistroCompletoTest.java

src/test/java/com/example/demo/service/RegistroCompletoTest.java

194 líneas originales. SHA-256 de revisión: a505133c3561ccda...

Dieciséis pruebas unitarias con repositorios simulados.

Líneas importantes

Línea 23. Inyecta mocks y configura respuestas de catálogo/guardado.

Línea 78. Comprueba mínimo uno y respuesta con IDs/apellido.

Línea 85. Comprueba cuatro contactos excluidos de la categoría titular.

Línea 93. Reutiliza una persona y conserva sus datos compartidos.

Línea 110. Quita vínculos sin borrar personas.

Línea 117. Rechaza contacto igual al titular.

Línea 122. Rechaza repetición de IDs en una lista.

Línea 142. Evita operar sobre contactos a través de las rutas de titular.

Línea 156. Comprueba el límite de altas.

Código numerado

```
1 | package com.example.demo.service;
2 |
3 | import com.example.demo.dto.*;
4 | import com.example.demo.model.*;
5 | import com.example.demo.repository.*;
6 | import org.junit.jupiter.api.BeforeEach;
7 | import org.junit.jupiter.api.Test;
8 | import org.springframework.test.util.ReflectionTestUtils;
9 | import org.springframework.web.server.ResponseStatusException;
10 | import java.time.LocalDate;
11 | import java.util.*;
12 | import static org.junit.jupiter.api.Assertions.*;
13 | import static org.mockito.Mockito.*;
14 |
15 | class RegistroCompletoTest {
16 |     private final PersonaRepository personas = mock(PersonaRepository.class);
17 |     private final CatalogoOcupacionRepository ocupaciones = mock(CatalogoOcupacionRepository.class);
18 |     private final CatalogoParentescoRepository parentescos =
19 |         mock(CatalogoParentescoRepository.class);
20 |     private final PersonaService service = new PersonaService();
21 |     private final CatalogoParentesco parentesco = new CatalogoParentesco();
22 |     private long siguienteId = 100;
23 |
24 |     @BeforeEach void configurar() {
25 |         ReflectionTestUtils.setField(service, "personaRepository", personas);
26 |         ReflectionTestUtils.setField(service, "ocupacionRepository", ocupaciones);
27 |         ReflectionTestUtils.setField(service, "parentescoRepository", parentescos);
28 |         parentesco.setId(9L);
29 |         parentesco.setNombre("Amigo(a)");
30 |         when(parentescos.findById(9L)).thenReturn(Optional.of(parentesco));
31 |         when(ocupaciones.findByName("Docente")).thenReturn(Optional.of(new CatalogoOcupacion()));
32 |         when(personas.save(any(Persona.class))).thenAnswer(i -> {
33 |             Persona p = i.getArgument(0);
34 |             if (p.getId() == null) p.setId(siguienteId++);
35 |             return p;
36 |         });
37 |         when(personas.saveAndFlush(any(Persona.class))).thenAnswer(i -> {
38 |             Persona p = i.getArgument(0);
39 |             if (p.getId() == null) p.setId(siguienteId++);
40 |             return p;
41 |         });
42 |     }
43 |
44 |     private ContactoDTO nuevo(String nombre) {
```

```

44 |         return new ContactoDTO(null, nombre, "PÃfÃrez", LocalDate.of(1980, 1, 1), "No
| especificado",
45 |             "contacto@example.com", "551111111", 9L, null);
46 |     }
47 |     private ContactoDTO existente(long id) {
48 |         return new ContactoDTO(id, null, null, null, 9L, null);
49 |     }
50 |     private FormularioDTO datos(int cantidad) {
51 |         FormularioDTO dto = new FormularioDTO();
52 |         dto.setNombre("Titular"); dto.setApellido("Prueba");
53 |         dto.setFechaNacimiento(LocalDate.of(1990, 1, 1));
54 |         dto.setEmail("principal@example.com"); dto.setTelefono("5512345678");
55 |         dto.setCorreosAdicionales(List.of("secundario@example.com"));
56 |         dto.setTelefonosAdicionales(List.of("5587654321"));
57 |         dto.setOcupacion("Docente");
58 |         dto.setContactosEmergencia(java.util.stream.IntStream.range(0, cantidad).mapToObj(i ->
| nuevo("Contacto " + i)).toList());
59 |         return dto;
60 |     }
61 |     private Persona persona(long id, boolean titular) {
62 |         Persona p = new Persona(); p.setId(id); p.setTitular(titular);
63 |         p.setNombre("Nombre " + id); p.setApellido("Apellido");
64 |         when(personas.findById(id)).thenReturn(Optional.of(p));
65 |         return p;
66 |     }
67 |     private ContactoEmergencia relacion(Persona titular, Persona contacto) {
68 |         ContactoEmergencia r = new ContactoEmergencia(); r.setPersona(titular);
69 |         r.setContacto(contacto); r.setParentesco(parentesco);
70 |         titular.getContactosEmergencia().add(r);
71 |         return r;
72 |     }
73 |
74 |     @Test void rechazaListaVacÃaAntesDeEscribir() {
75 |         assertThrows(ResponseStatusException.class, () -> service.guardar(datos(0)));
76 |         verify(personas, never()).save(any()); verify(personas, never()).saveAndFlush(any());
77 |     }
78 |     @Test void permiteUnContactoYDevuelveIdentidadCompleta() {
79 |         FormularioDTO resultado = service.guardar(datos(1));
80 |         assertEquals(1, resultado.getContactosEmergencia().size());
81 |         assertNotNull(resultado.getContactosEmergencia().getFirst().idContacto());
82 |         assertEquals("PÃfÃrez", resultado.getContactosEmergencia().getFirst().apellido());
83 |         assertEquals("Amigo(a)", resultado.getContactosEmergencia().getFirst().parentesco());
84 |     }
85 |     @Test void permiteMasDeDosContactosSinContarlosComoTitulares() {
86 |         FormularioDTO resultado = service.guardar(datos(4));
87 |         assertEquals(4, resultado.getContactosEmergencia().size());
88 |         verify(personas, times(4)).save(argThat(p -> !p.isTitular() && p.getTelefonos().size() ==
| 1));
89 |         verify(personas, times(2)).saveAndFlush(argThat(p -> p.isTitular() && p.getCorreos().size()
| == 2));
90 |         verify(personas).countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull();
91 |         assertEquals(List.of("secundario@example.com"), resultado.getCorreosAdicionales());
92 |     }
93 |     @Test void compartePersonaSinModificarSusDatosPersonales() {
94 |         Persona a = persona(1, true), b = persona(2, true), c = persona(3, false);
95 |         c.setNombre("Nombre original");
96 |         c.setApellido("Apellido original");
97 |         c.setFechaNacimiento(LocalDate.of(1980, 1, 1));
98 |         c.setGenero("No especificado");
99 |         service.guardarContactos(1L, List.of(existente(3)));
100 |         service.guardarContactos(2L, List.of(new ContactoDTO(3L, "No sobrescribir", "Otro",
|             LocalDate.of(2000, 1, 1), "Otro", "otro@example.com", "000000000", 9L, null)));
101 |         assertSame(c, a.getContactosEmergencia().getFirst().getContacto());
102 |         assertSame(c, b.getContactosEmergencia().getFirst().getContacto());
103 |         assertEquals("Nombre original", c.getNombre());
104 |         assertEquals("Apellido original", c.getApellido());
105 |         assertEquals(LocalDate.of(1980, 1, 1), c.getFechaNacimiento());
106 |         assertEquals("No especificado", c.getGenero());
107 |         verify(personas, never()).save(c);
108 |     }
109 |
110 |     @Test void reemplazarQuitaSoloRelacionYSostieneLasConservadas() {
111 |         Persona a = persona(1, true), b = persona(2, false), c = persona(3, false);
112 |         relacion(a,b); ContactoEmergencia conservada = relacion(a,c);
113 |         service.guardarContactos(1L, List.of(existente(3)));
114 |         assertEquals(List.of(conservada), a.getContactosEmergencia());
115 |         verify(personas, never()).delete(any());
116 |     }
117 |     @Test void rechazaAutorreferencia() {
118 |         persona(1,true);
119 |         assertThrows(ResponseStatusException.class, () ->
| service.guardarContactos(1L,List.of(existente(1))));

```

```

120 |         verify(personas, never()).saveAndFlush(any());
121 |     }
122 |     @Test void rechazaIDsDuplicados() {
123 |         assertThrows(ResponseStatusException.class, () ->
124 |         service.guardarContactos(1L,List.of(existente(3),existente(3))));
125 |         verify(personas, never()).findById(any());
126 |     }
127 |     @Test void rechazaPersonaInexistenteOBaja() {
128 |         persona(1,true); persona(3,false).setFechaBaja(LocalDate.now());
129 |         assertThrows(ResponseStatusException.class, () ->
130 |         service.guardarContactos(1L,List.of(existente(99))));
131 |         assertThrows(ResponseStatusException.class, () ->
132 |         service.guardarContactos(1L,List.of(existente(3))));
133 |     }
134 |     @Test void rechazaParentescoInexistente() {
135 |         persona(1,true);
136 |         assertThrows(ResponseStatusException.class, () -> service.guardarContactos(1L,
137 |         List.of(new ContactoDTO(null,"Ana","PÃfÃorez","551111111",99L,null)));
138 |         verify(personas, never()).save(any());
139 |     }
140 |     @Test void exigeApellidoParaNuevaPersonaContacto() {
141 |         var dto = datos(1);
142 |         dto.setContactosEmergencia(List.of(new ContactoDTO(null,"Ana",null,"551111111",9L,null)));
143 |         assertThrows(ResponseStatusException.class, () -> service.guardar(dto));
144 |     }
145 |     @Test void contactoNoSePuedeEditarComoTitular() {
146 |         persona(3,false);
147 |         assertEquals(404, assertThrows(ResponseStatusException.class, () ->
148 |         service.actualizar(3L,datos(1))).getStatusCode().value());
149 |         assertThrows(ResponseStatusException.class, () -> service.eliminarLogico(3L));
150 |     }
151 |     @Test void listaSoloTitulares() {
152 |         Persona titular = persona(1,true);
153 |         CatalogoOcupacion ocupacion = new CatalogoOcupacion();
154 |         ocupacion.setNombre("Docente");
155 |         titular.getPerfilTitular().setOcupacion(ocupacion);
156 |         when(personas.findByIdPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull()).thenReturn(List
157 |         .of(titular));
158 |         assertEquals(1,service.obtenerTodos().size());
159 |         verify(personas).findByIdPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull();
160 |     }
161 |     @Test void limiteVeinteTitulares() {
162 |         when(personas.countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull()).thenReturn(20L);
163 |         assertThrows(RuntimeException.class, () -> service.guardar(datos(1)));
164 |         verify(personas, never()).save(any());
165 |     }
166 |     @Test void reactivaTitularDesactivadoSiHayCupo() {
167 |         Persona desactivada = persona(4, true);
168 |         desactivada.getPerfilTitular().setFechaBaja(LocalDate.now());
169 |         CatalogoOcupacion ocupacion = new CatalogoOcupacion();
170 |         ocupacion.setNombre("Docente");
171 |         desactivada.getPerfilTitular().setOcupacion(ocupacion);
172 |         when(personas.countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull()).thenReturn(19L);
173 |         FormularioDTO resultado = service.reactivar(4L);
174 |         assertNull(desactivada.getPerfilTitular().getFechaBaja());
175 |         assertEquals(4L, resultado.getId());
176 |         verify(personas).save(desactivada);
177 |     }
178 |     @Test void rechazaReactivacionCuandoSeAlcanzoElLimite() {
179 |         Persona desactivada = persona(4, true);
180 |         desactivada.getPerfilTitular().setFechaBaja(LocalDate.now());
181 |         when(personas.countByPerfilTitularIsNotNullAndPerfilTitularFechaBajaIsNull()).thenReturn(20L);
182 |         ResponseStatusException error = assertThrows(ResponseStatusException.class, () ->
183 |         service.reactivar(4L));
184 |         assertEquals(409, error.getStatusCode().value());
185 |         assertNotNull(desactivada.getPerfilTitular().getFechaBaja());
186 |         verify(personas, never()).save(any());
187 |     }
188 |     @Test void rechazaTelefonoSecundarioInvalido() {
189 |         FormularioDTO dto=datos(1); dto.setTelefonosAdicionales(List.of("123"));
190 |         assertThrows(IllegalArgumentException.class, () -> service.guardar(dto));
191 |         verify(personas, never()).save(any());

```

```
192 |  
193 | }  
194 |
```

27. DemoApplicationTests.java

src/test/java/com/example/demo/DemoApplicationTests.java

103 líneas originales. SHA-256 de revisión: 8890892656d068e8...

Siete pruebas de integración con PostgreSQL. Sustituyen la suite anterior que dependía de AuthService inexistente.

Líneas importantes

Línea 18. Solo habilita una base local desechable nomina_backend_test con puerto explícito.

Línea 24. Prueba contra el esquema SQL real, no contra uno inventado por Hibernate.

Línea 32. Crea tablas si faltan y vacía datos de la base de prueba.

Línea 51. Comprueba la persistencia de cuatro personas de contacto y un titular.

Línea 60. Verifica que borrar un vínculo no borra al contacto compartido.

Línea 71. Comprueba repetición segura cuando se conservan los IDs.

Línea 77. Comprueba rollback del alta completa.

Línea 84. Comprueba rollback durante sustitución de contactos.

Línea 91. Verifica CHECK y UNIQUE en PostgreSQL.

Línea 98. Baja lógica sin borrado de personas.

Código numerado

```
1 | package com.example.demo;
2 |
3 | import com.example.demo.dto.*;
4 | import com.example.demo.service.PersonaService;
5 | import org.junit.jupiter.api.BeforeEach;
6 | import org.junit.jupiter.api.Test;
7 | import org.junit.jupiter.api.condition.EnabledIfEnvironmentVariable;
8 | import org.springframework.beans.factory.annotation.Autowired;
9 | import org.springframework.boot.test.context.SpringBootTest;
10 | import org.springframework.jdbc.core.JdbcTemplate;
11 | import org.springframework.core.io.ClassPathResource;
12 | import org.springframework.web.server.ResponseStatusException;
13 | import java.nio.charset.StandardCharsets;
14 | import java.time.LocalDate;
15 | import java.util.List;
16 | import static org.junit.jupiter.api.Assertions.*;
17 |
18 | @EnabledIfEnvironmentVariable(named = "TEST_DATABASE_URL",
19 |     matches = "jdbc:postgresql://(localhost|127\\.0\\.0\\.1):[0-9]+/nomina_backend_test")
20 | @SpringBootTest(properties = {
21 |     "spring.datasource.url=${TEST_DATABASE_URL}",
22 |     "spring.datasource.username=${TEST_DATABASE_USERNAME:postgres}",
23 |     "spring.datasource.password=${TEST_DATABASE_PASSWORD:}",
24 |     "spring.jpa.hibernate.ddl-auto=none",
25 |     "spring.sql.init.mode=never",
26 |     "jwt.secret=test-only-secret-with-at-least-32-bytes"
27 | })
28 | class DemoApplicationTests {
29 |     @Autowired PersonaService personas;
30 |     @Autowired JdbcTemplate jdbc;
31 |
32 |     @BeforeEach void prepararBaseDesechable() throws Exception {
33 |         jdbc.execute(new
34 |             ClassPathResource("schema.sql").getContentAsString(StandardCharsets.UTF_8));
35 |         jdbc.execute("TRUNCATE TABLE persona, catalogo_ocupacion, catalogo_parentesco RESTART
36 |             IDENTITY CASCADE");
37 |         jdbc.update("INSERT INTO catalogo_parentesco(id,nombre) VALUES (1,'Amigo')");
38 |     }
39 |
40 |     private FormularioDTO formulario(int cantidad) {
41 |         FormularioDTO d = new FormularioDTO();
```

```

39 |         d.setNombre("Titular"); d.setApellido("Prueba");
40 |         d.setFechaNacimiento(LocalDate.of(1990,1,1)); d.setOcupacion("Docente");
41 |         d.setEmail("uno@example.com"); d.setCorreosAdicionales(List.of("dos@example.com"));
42 |         d.setTelefono("5511111111"); d.setTelefonosAdicionales(List.of("5522222222"));
43 |         d.setContactosEmergencia(java.util.stream.IntStream.range(0,cantidad)
44 |             .mapToObj(i -> new ContactoDTO(null, "Contacto " + i, "Apellido", LocalDate.of(1980,
45 | 1, 1),
46 |                 "No especificado", "contacto@example.com", "5533333333", 1L, null)).toList());
47 |         return d;
48 |     }
49 |     private ContactoDTO existente(long id) { return new ContactoDTO(id,null,null,null,1L,null); }
50 |     private int count(String table) { return jdbc.queryForObject("SELECT count(*) FROM " +
51 | table,Integer.class); }
52 |
53 |     @Test void persisteCuatroContactosComoPersonasYListaSoloTitular() {
54 |         var d = personas.guardar(formulario(4));
55 |         assertEquals(5,count("persona"));
56 |         assertEquals(4,count("persona_contacto_emergencia"));
57 |         assertEquals(1,personas.obtenerTodos().size());
58 |         assertEquals(4,personas.obtenerContactos(d.getId()).size());
59 |         assertTrue(jdbc.queryForObject("SELECT es_titular FROM persona WHERE
60 | id=?",Boolean.class,d.getId()));
61 |         assertEquals(4,jdbc.queryForObject("SELECT count(*) FROM persona WHERE NOT
62 | es_titular",Integer.class));
63 |     }
64 |     @Test void contactoCompartidoSobreviveARetirarUnaRelacion() {
65 |         var a=personas.guardar(formulario(2));
66 |         Long compartido=a.getContactosEmergencia().getFirst().idContacto();
67 |         var otro=formulario(1); otro.setContactosEmergencia(List.of(existente(compartido)));
68 |         var b=personas.guardar(otro);
69 |         assertEquals(4,count("persona"));
70 |         personas.guardarContactos(a.getId(),List.of(existente(a.getContactosEmergencia().get(1).idCo
71 | ntacto())));
72 |         assertEquals(4,count("persona"));
73 |         assertEquals(compartido,personas.obtenerContactos(b.getId()).getFirst().idContacto());
74 |         assertEquals(2,count("persona_contacto_emergencia"));
75 |     }
76 |     @Test void repetirPutConIDsNoDuplicaPersonasNiRelaciones() {
77 |         var d=personas.guardar(formulario(3));
78 |         personas.guardarContactos(d.getId(),d.getContactosEmergencia());
79 |         personas.guardarContactos(d.getId(),d.getContactosEmergencia());
80 |         assertEquals(4,count("persona")); assertEquals(3,count("persona_contacto_emergencia"));
81 |     }
82 |     @Test void falloPosteriorRevierdeNuevasPersonasYOcupacion() {
83 |         var d=formulario(2);
84 |         d.setContactosEmergencia(List.of(d.getContactosEmergencia().getFirst(),existente(99999)));
85 |         assertThrows(ResponseStatusException.class,()->personas.guardar(d));
86 |         assertEquals(0,count("persona")); assertEquals(0,count("catalogo_ocupacion"));
87 |         assertEquals(0,count("persona_telefono"));
88 |     }
89 |     @Test void falloPutRevierdeCambioDeRelaciones() {
90 |         var d=personas.guardar(formulario(2));
91 |         assertThrows(ResponseStatusException.class,()->personas.guardarContactos(d.getId(),
92 |             List.of(new ContactoDTO(null, "Nuevo", "Apellido", LocalDate.of(1980, 1, 1),
93 |                 "No especificado", "nuevo@example.com", "5544444444", 1L, null),
94 | existente(99999))));
95 |         assertEquals(3,count("persona"));
96 |         assertEquals(2,personas.obtenerContactos(d.getId()).size());
97 |     }
98 |     @Test void restriccionesSqlRechazanAutorreferenciaYDuplicado() {
99 |         var d=personas.guardar(formulario(1));
100 |         assertThrows(org.springframework.dao.DataIntegrityViolationException.class,()->jdbc.update(
101 |             "INSERT INTO persona_contacto_emergencia(id_persona,id_contacto,id_parentesco)
102 | VALUES (?,?,1)",d.getId(),d.getId()));
103 |         assertThrows(org.springframework.dao.DataIntegrityViolationException.class,()->jdbc.update(
104 |             "INSERT INTO persona_contacto_emergencia(id_persona,id_contacto,id_parentesco)
105 | VALUES (?,?,1)",d.getId(),d.getContactosEmergencia().getFirst().idContacto());
106 |     }
107 |     @Test void bajaTitularNoEliminaContactoCompartido() {
108 |         var a=personas.guardar(formulario(1));
109 |         personas.eliminarLogico(a.getId());
110 |         assertEquals(0,personas.obtenerTodos().size()); assertEquals(2,count("persona"));
111 |     }
112 | }

```

28. maven-wrapper.properties

.mvn/wrapper/maven-wrapper.properties

3 líneas originales. SHA-256 de revisión: 488e1b3f2e641779...

Distribución Maven usada por los lanzadores.

Líneas importantes

Línea 1. Versión del wrapper.

Línea 2. Modo only-script.

Línea 3. URL de la distribución Maven.

Código numerado

```
1 | wrapperVersion=3.3.4
2 | distributionType=only-script
3 | distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.16/apache-
  | maven-3.9.16-bin.zip
```

29. mvnw.cmd

mvnw.cmd

189 líneas originales. SHA-256 de revisión: 46eedb8419bd14fe...

Lanzador Windows: una entrada batch ejecuta PowerShell para localizar o descargar Maven y pasarle argumentos.

Líneas importantes

Línea 35. Ejecuta la sección PowerShell del archivo.

Línea 54. Lee la URL de propiedades del wrapper.

Línea 102. Reutiliza distribución existente en caché.

Línea 135. Descarga Maven si falta.

Línea 138. Lee checksum opcional y lo verifica si está configurado.

Línea 150. Extrae la distribución descargada.

Línea 104. Comunica la ruta de ejecución a batch.

Extractos numerados

```
35 | @FOR /F "usebackq tokens=1* delims==" %%A IN (`powershell -nopprofile "& {$scriptDir='%~dp0';  
    | $script='%__MVNW_ARG0_NAME__'; icm -ScriptBlock ([Scriptblock]::Create((Get-Content -Raw '%~f0')))  
    | -NoNewScope}`) DO @(  
36 |     IF "%%A"=="MVN_CMD" (set __MVNW_CMD__=%%B) ELSE IF "%%B"==" " (echo %%A) ELSE (echo %%A=%%B)  
37 | )  
38 | @SET PSModulePath=%__MVNW_PSMODULEP_SAVE%  
...  
54 | $distributionUrl = (Get-Content -Raw "$scriptDir/.mvn/wrapper/maven-wrapper.properties" |  
    | ConvertFrom-StringData).distributionUrl  
55 | if (!$distributionUrl) {  
56 |     Write-Error "cannot read distributionUrl property in $scriptDir/.mvn/wrapper/maven-  
    | wrapper.properties"  
57 | }  
...  
102 | if (Test-Path -Path "$MAVEN_HOME" -PathType Container) {  
103 |     Write-Verbose "found existing MAVEN_HOME at $MAVEN_HOME"  
104 |     Write-Output "MVN_CMD=$MAVEN_HOME/bin/$MVN_CMD"  
105 |     exit $?  
106 | }  
107 |  
...  
135 | $webclient.DownloadFile($distributionUrl, "$TMP_DOWNLOAD_DIR/$distributionUrlName") | Out-Null  
136 |  
137 | # If specified, validate the SHA-256 sum of the Maven distribution zip file  
138 | $distributionSha256Sum = (Get-Content -Raw "$scriptDir/.mvn/wrapper/maven-wrapper.properties" |  
    | ConvertFrom-StringData).distributionSha256Sum  
139 | if ($distributionSha256Sum) {  
140 |     if ($USE_MVND) {  
141 |         Write-Error "Checksum validation is not supported for maven-mvnd. `nPlease disable validation by  
    | removing 'distributionSha256Sum' from your maven-wrapper.properties."  
...  
150 | Expand-Archive "$TMP_DOWNLOAD_DIR/$distributionUrlName" -DestinationPath "$TMP_DOWNLOAD_DIR" | Out-  
    | Null  
151 |  
152 | # Find the actual extracted directory name (handles snapshots where filename != directory name)  
153 | $actualDistributionDir = ""
```

30. mvnw

mvnw

295 líneas originales. SHA-256 de revisión: cae96cef89e8ea35...

Lanzador para Unix y shells compatibles: descubre Java, reutiliza o descarga Maven y ejecuta sus argumentos.

Líneas importantes

Línea 45. Localiza Java.

Línea 117. Lee configuración del wrapper.

Línea 148. Función que invoca Maven.

Línea 153. Detecta distribución ya instalada.

Línea 194. Selecciona descarga por wget; después considera curl y Java.

Línea 226. Verifica checksum solo si está configurado.

Línea 155. Invoca Maven; la primera coincidencia pertenece a la rama de caché.

Extractos numerados

```
45 | set_java_home() {
46 |     # For Cygwin and MinGW, ensure paths are in Unix format before anything is touched
47 |     if [ -n "${JAVA_HOME-}" ]; then
48 |         if [ -x "$JAVA_HOME/jre/sh/java" ]; then
...
117 | done <"$scriptDir/.mvn/wrapper/maven-wrapper.properties"
118 | [ -n "${distributionUrl-}" ] || die "cannot read distributionUrl property in
    | $scriptDir/.mvn/wrapper/maven-wrapper.properties"
119 |
120 | case "${distributionUrl##*/}" in
...
148 | exec_maven() {
149 |     unset MVNW_VERBOSE MVNW_USERNAME MVNW_PASSWORD MVNW_REPOURL || :
150 |     exec "$MAVEN_HOME/bin/$MVNW_CMD" "$@" || die "cannot exec $MAVEN_HOME/bin/$MVNW_CMD"
151 | }
...
153 | if [ -d "$MAVEN_HOME" ]; then
154 |     verbose "found existing MAVEN_HOME at $MAVEN_HOME"
155 |     exec_maven "$@"
156 | fi
157 |
158 | case "${distributionUrl-}" in
...
194 | if [ -z "${MVNW_USERNAME-}" ] && command -v wget >/dev/null; then
195 |     verbose "Found wget ... using wget"
196 |     wget "${MVNW_QUIET_WGET:+"$MVNW_QUIET_WGET"}" "$distributionUrl" -O
    | "$TMP_DOWNLOAD_DIR/$distributionUrlName" || die "wget: Failed to fetch $distributionUrl"
197 | elif [ -z "${MVNW_USERNAME-}" ] && command -v curl >/dev/null; then
...
226 | if [ -n "${distributionSha256Sum-}" ]; then
227 |     distributionSha256Result=false
228 |     if [ "$MVNW_CMD" = mvnd.sh ]; then
229 |         echo "Checksum validation is not supported for maven-mvnd." >&2
```

31. .gitignore

.gitignore

33 líneas originales. SHA-256 de revisión: 089382e4d7944ab6...

Exclusiones para archivos locales y generados; no elimina archivos ya versionados.

Líneas importantes

Línea 2. Excluye resultados de compilación.

Línea 3. Excluye el binario del wrapper.

Línea 17. Excluye configuración IntelliJ.

Línea 33. Excluye configuración VS Code.

Código numerado

```
1 | HELP.md
2 | target/
3 | .mvn/wrapper/maven-wrapper.jar
4 | !**/src/main/**/target/
5 | !**/src/test/**/target/
6 |
7 | ### STS ###
8 | .apt_generated
9 | .classpath
10 | .factorypath
11 | .project
12 | .settings
13 | .springBeans
14 | .sts4-cache
15 |
16 | ### IntelliJ IDEA ###
17 | .idea
18 | *.iws
19 | *.iml
20 | *.ipr
21 |
22 | ### NetBeans ###
23 | /nbproject/private/
24 | /nbbuild/
25 | /dist/
26 | /nbdist/
27 | /.nb-gradle/
28 | build/
29 | !**/src/main/**/build/
30 | !**/src/test/**/build/
31 |
32 | ### VS Code ###
33 | .vscode/
```

32. .gitattributes

.gitattributes

3 líneas originales. SHA-256 de revisión: 1c8cc8fb65937e0e...

Reglas de Git para finales de línea.

Líneas importantes

Línea 1. Fuerza LF en el lanzador Unix.

Línea 2. Fuerza CRLF en comandos Windows.

Código numerado

```
1 | /mvnw text eol=lf
2 | *.cmd text eol=crlf
3 | *.pdf binary
```

33. README.md

README.md

625 líneas originales. SHA-256 de revisión: f443207ad643896c...

Contrato actual de API y guía de archivos.

Líneas importantes

Línea 7. Modelo de titulares, contactos y relaciones.

Línea 17. Arranque y conexión.

Línea 27. Token nuevo de una hora y alcance del login.

Línea 33. Endpoints incluyendo catálogo de parentescos.

Línea 52. JSON nuevo, IDs y sustitución de listas.

Línea 97. Preparación de una base nueva.

Línea 113. Resultados y ejecución en base desechable.

Línea 131. Regeneración y alcance.

Extractos numerados

```
7 | ## Funcionamiento
8 |
9 | El frontend envía JSON al controlador; PersonaService valida y coordina transacciones; los
  | repositorios persisten entidades. Un contacto es una fila de `persona`, y
  | `persona_contacto_emergencia` une al titular con esa persona y un parentesco de
  | `catalogo_parentesco`.
10 |
...
17 | ## Configuración
18 |
19 | La configuración local de PostgreSQL y JWT está en `src/main/resources/application.properties`. **El
  | archivo contiene valores sensibles escritos directamente**: no los copies a documentación, logs o
  | ejemplos; externalízalos antes de desplegar o compartir el proyecto y rota los secretos si ya fueron
  | expuestos. No se debe asumir que las variables de entorno ya están conectadas a la configuración
  | actual. CORS admite `http://localhost:4200`.
20 |
...
27 | ## Autenticación
28 |
29 | POST `/api/auth/login` compara usuario y BCrypt, y devuelve `message`, `username` y `token`. Los
  | **tokens nuevos duran una hora** (3600000 ms). Los tokens ya emitidos conservan su vencimiento
  | original.
30 |
...
33 | ## Rutas
34 |
35 | | Método y ruta | Función |
36 | | --- | --- |
...
52 | ## Registro y actualización
53 |
54 | El titular necesita nombre, apellido, fecha de nacimiento, ocupación al crear, dos correos y dos
  | teléfonos. Las reglas de comunicaciones existentes se mantienen. Los contactos nuevos requieren
  | nombre, apellido, fecha de nacimiento, género, correo y un teléfono de diez dígitos, de acuerdo con
  | las restricciones del esquema.
55 |
...
97 | ## SQL inicial
98 |
99 | Ejecuta primero [schema.sql](src/main/resources/schema.sql) y después
  | [data.sql](src/main/resources/data.sql) sobre una base vacía. Estos scripts ya contienen el modelo
  | actual: `catalogo_parentesco`, `persona.es_titular` y `persona_contacto_emergencia`.
```

```
100 |  
...  
113 | ## Pruebas existentes  
114 |  
115 | - RegistroCompletoTest: 14 pruebas unitarias de listas variables, contactos compartidos, validación,  
    | distinción de titulares y límite.  
116 | - RegistroCompletoTest: 16 pruebas unitarias de listas variables, contactos compartidos, validación,  
    | distinción de titulares y reactivación.  
...  
131 | ## Documentación PDF  
132 |  
133 | El generador calcula referencias de líneas, verifica cobertura y omite secretos en el PDF. Requiere  
    | Python, reportlab, pypdf y fuentes Arial/Consolas de Windows.  
134 |
```

34. CatalogoParentesco.java

src/main/java/com/example/demo/model/CatalogoParentesco.java

19 líneas originales. SHA-256 de revisión: 976713f9536a4e7b...

Entidad del catálogo de parentescos.

Líneas importantes

Línea 6. Tabla catalogo_parentesco.

Línea 9. ID generado.

Línea 12. Nombre obligatorio, único y de hasta 50 caracteres.

Código numerado

```
1 | package com.example.demo.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "catalogo_parentesco")
7 | public class CatalogoParentesco {
8 |     @Id
9 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
10 |    private Long id;
11 |
12 |    @Column(nullable = false, unique = true, length = 50)
13 |    private String nombre;
14 |
15 |    public Long getId() { return id; }
16 |    public void setId(Long id) { this.id = id; }
17 |    public String getNombre() { return nombre; }
18 |    public void setNombre(String nombre) { this.nombre = nombre; }
19 | }
```

35. CatalogoParentescoRepository.java

src/main/java/com/example/demo/repository/CatalogoParentescoRepository.java

9 líneas originales. SHA-256 de revisión: 60458d506b3087ba...

Acceso al catálogo por ID o nombre.

Líneas importantes

Línea 7. Hereda operaciones de catálogo.

Línea 8. Busca un nombre exacto para resolver el vínculo.

Código numerado

```
1 | package com.example.demo.repository;
2 |
3 | import com.example.demo.model.CatalogoParentesco;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import java.util.Optional;
6 |
7 | public interface CatalogoParentescoRepository extends JpaRepository<CatalogoParentesco, Long> {
8 |     Optional<CatalogoParentesco> findByNombre(String nombre);
9 | }
```

36. ParentescoController.java

src/main/java/com/example/demo/controller/ParentescoController.java

22 líneas originales. SHA-256 de revisión: 7aa9c5c58cbebc1a...

Expone el catálogo al frontend.

Líneas importantes

Línea 9. Ruta protegida /api/parentescos.

Línea 18. Consulta HTTP GET.

Línea 20. Devuelve catálogo ordenado por ID.

Código numerado

```
1 | package com.example.demo.controller;
2 |
3 | import com.example.demo.model.CatalogoParentesco;
4 | import com.example.demo.repository.CatalogoParentescoRepository;
5 | import org.springframework.web.bind.annotation.*;
6 | import java.util.List;
7 |
8 | @RestController
9 | @RequestMapping("/api/parentescos")
10 | @CrossOrigin(origins = "http://localhost:4200")
11 | public class ParentescoController {
12 |     private final CatalogoParentescoRepository parentescos;
13 |
14 |     public ParentescoController(CatalogoParentescoRepository parentescos) {
15 |         this.parentescos = parentescos;
16 |     }
17 |
18 |     @GetMapping
19 |     public List<CatalogoParentesco> obtenerTodos() {
20 |         return parentescos.findAll(org.springframework.data.domain.Sort.by("id"));
21 |     }
22 | }
```

37. JwtUtilTest.java

src/test/java/com/example/demo/security/JwtUtilTest.java

21 líneas originales. SHA-256 de revisión: b4fa3c0d32d70d99...

Prueba de duración del token y sujeto.

Líneas importantes

Línea 11. Genera JWT y analiza sus fechas y usuario.

Línea 18. Verifica una hora exacta entre emisión y vencimiento.

Código numerado

```
1 | package com.example.demo.security;
2 |
3 | import io.jsonwebtoken.Jwts;
4 | import io.jsonwebtoken.security.Keys;
5 | import org.junit.jupiter.api.Test;
6 | import org.springframework.test.util.ReflectionTestUtils;
7 | import java.nio.charset.StandardCharsets;
8 | import static org.junit.jupiter.api.Assertions.*;
9 |
10 | class JwtUtilTest {
11 |     @Test void tokenDuraUnaHoraYConservaUsuario() {
12 |         String secret = "test-only-secret-with-at-least-32-bytes";
13 |         JwtUtil jwt = new JwtUtil();
14 |         ReflectionTestUtils.setField(jwt, "secretKey", secret);
15 |         String token = jwt.generateToken("admin");
16 |         var claims =
17 |             Jwts.parserBuilder().setSigningKey(Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)))
18 |                 .build().parseClaimsJws(token).getBody();
19 |         assertEquals(3600000L, claims.getExpiration().getTime() - claims.getIssuedAt().getTime());
20 |         assertEquals("admin", jwt.extractUsername(token));
21 |     }
```